

CSE 6220/CX 4220

Introduction to HPC

Lecture 5: Caching and Cache-Efficient Algorithms

Helen Xu

hxu615@gatech.edu



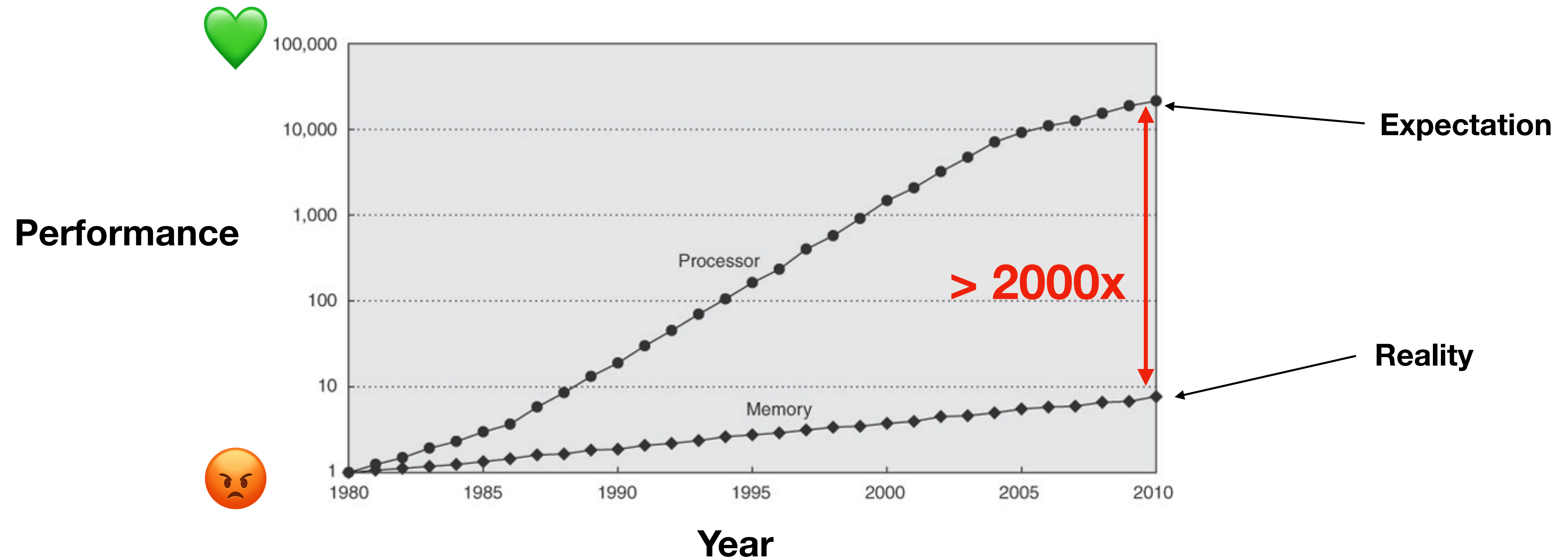
Georgia Tech College of Computing
School of Computational
Science and Engineering

**Let's discuss the memory hierarchy
on one processor - why?**

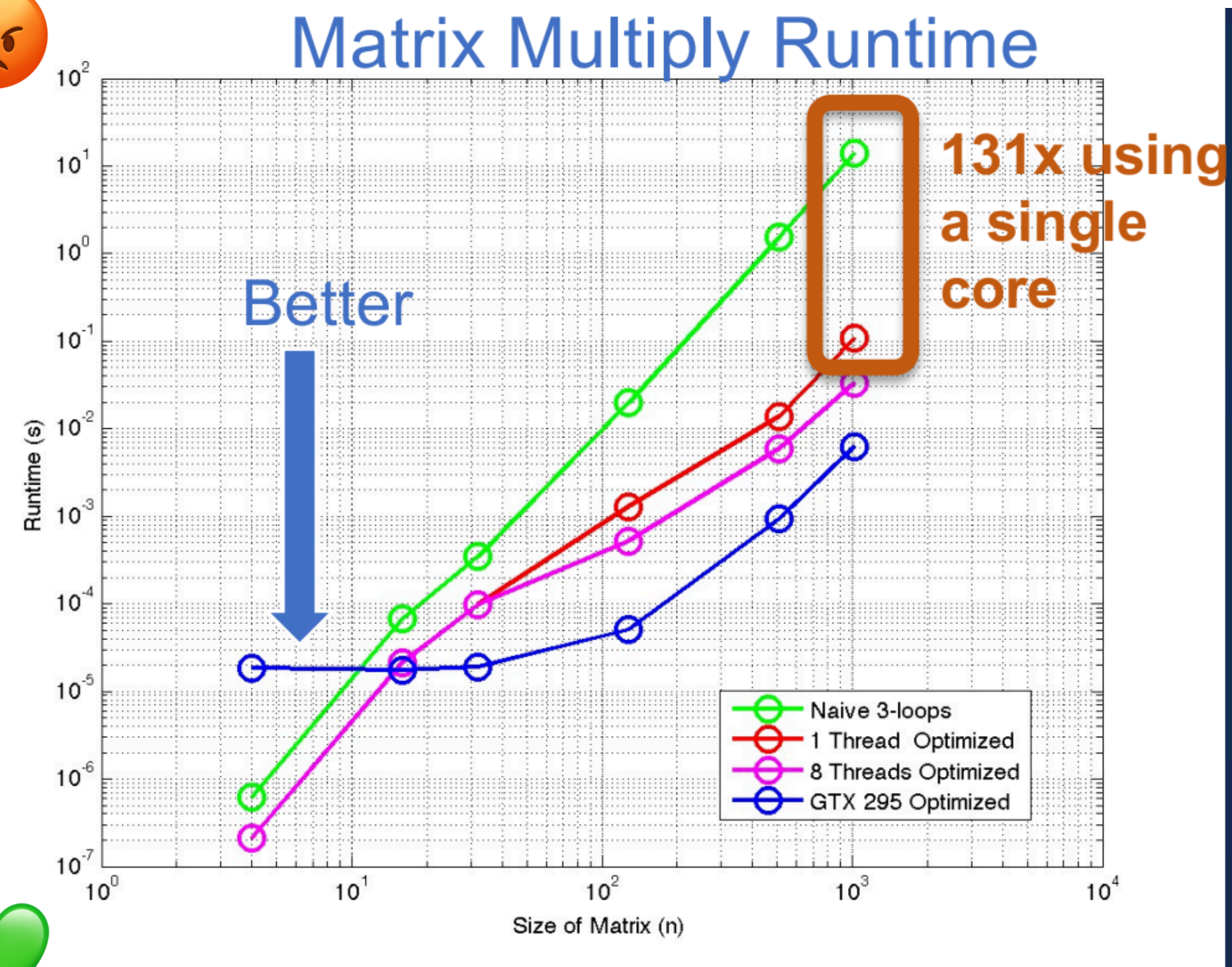
Memory limitations affect overall performance

Most applications run at **<10%** of “peak” performance (max possible rate of flops).

Much of the performance is lost on a single processor due to **data movement**.



Why should we care about serial performance in a course on high-performance computing?



Starting with a fast serial implementation is an **important first step** towards a fast parallel implementation.

Another way to say it: **If you parallelize slow serial code, you will get slow parallel code.**

Matrix multiply is extreme, but ~10x improvement from **sequential optimizations** is more common.

Memory Hierarchies

Memory access performance measurements

Memory accesses (load/store) have two costs:

Latency - the cost to load or store one word (τ)

Bandwidth - the average rate (bytes / sec) to load/store a large chunk of data (μ)

Bandwidth

$\approx$ data throughput (bytes/second)



Low Bandwidth



High Bandwidth

Latency

$\approx$ delay due data travel time (secs)

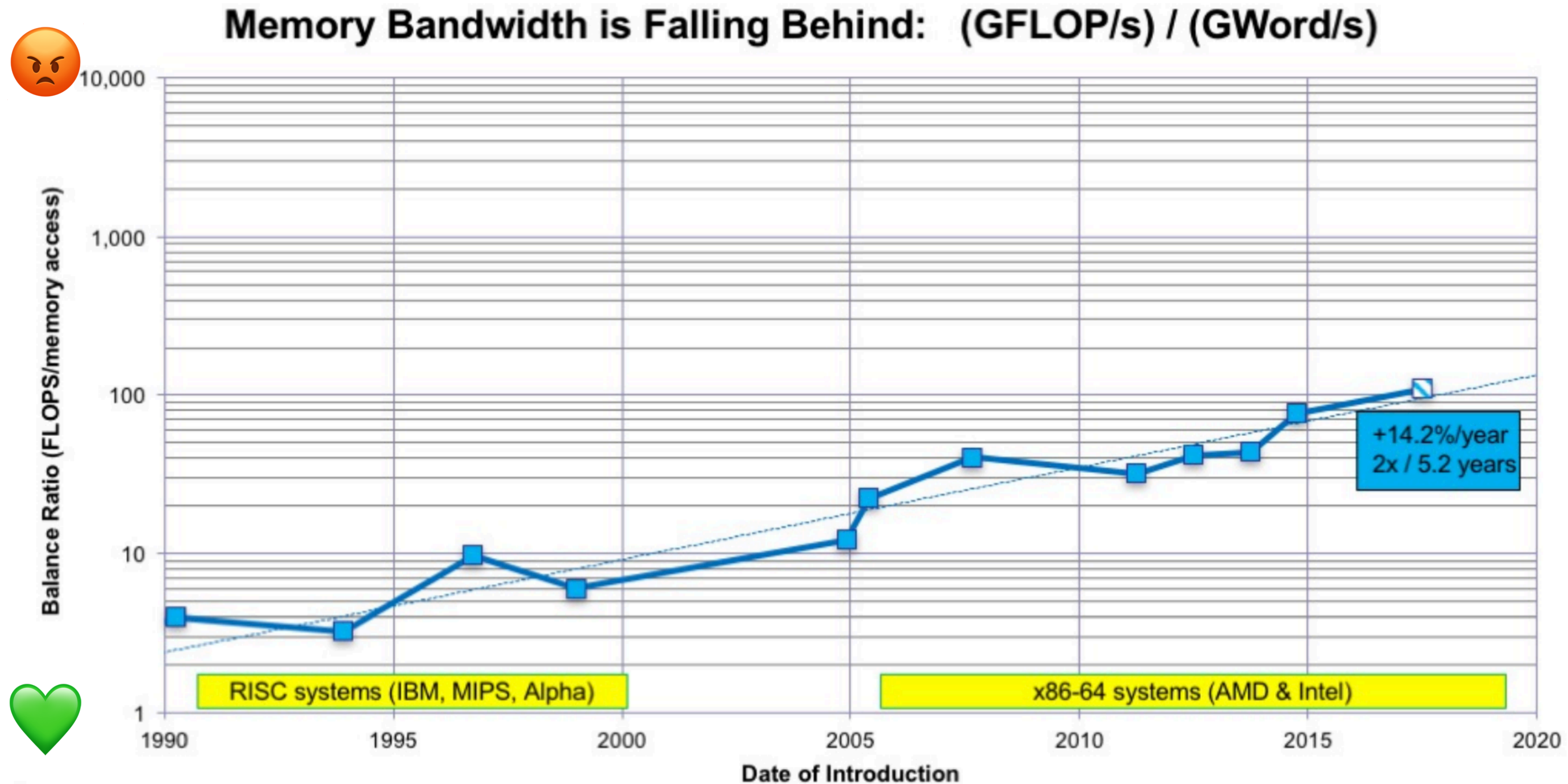


Low Latency



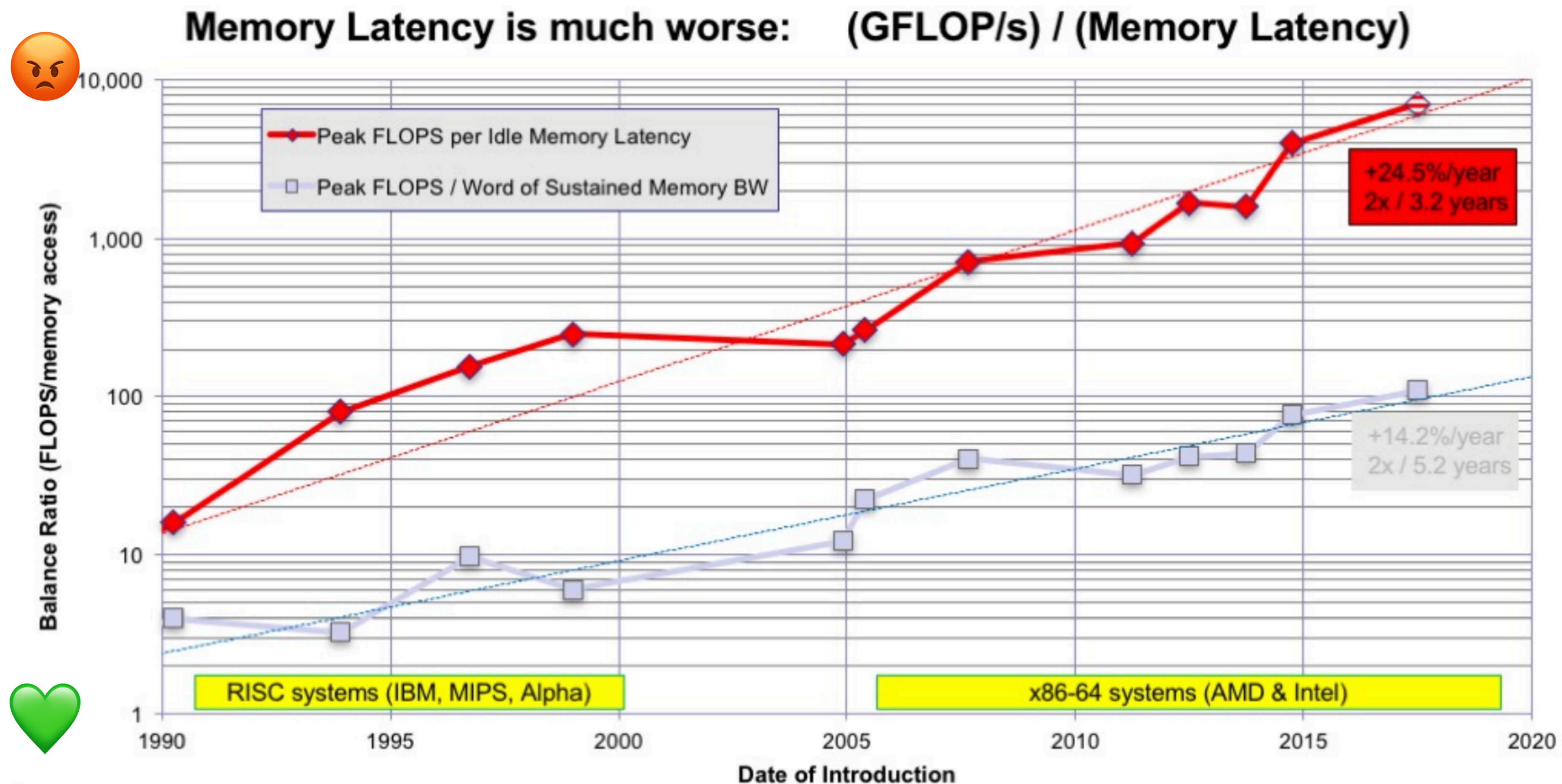
High Latency

Memory bandwidth gap



THE UNIVERSITY OF TEXAS AT AUSTIN
TEXAS ADVANCED COMPUTING CENTER

Memory latency gap is worse

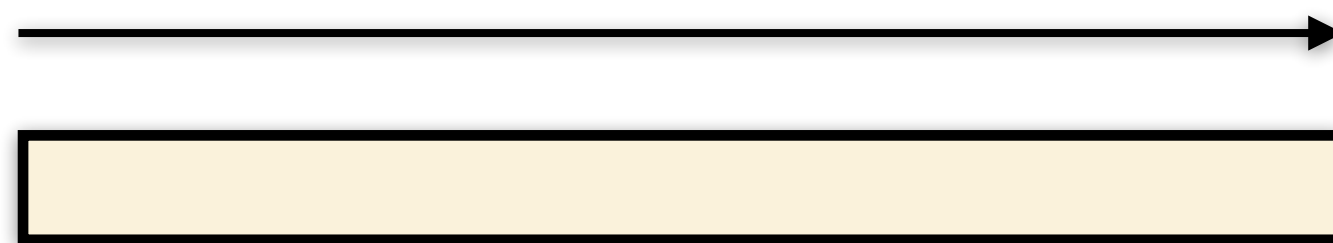


THE UNIVERSITY OF TEXAS AT AUSTIN
TEXAS ADVANCED COMPUTING CENTER

Two main types of locality: Spatial and Temporal

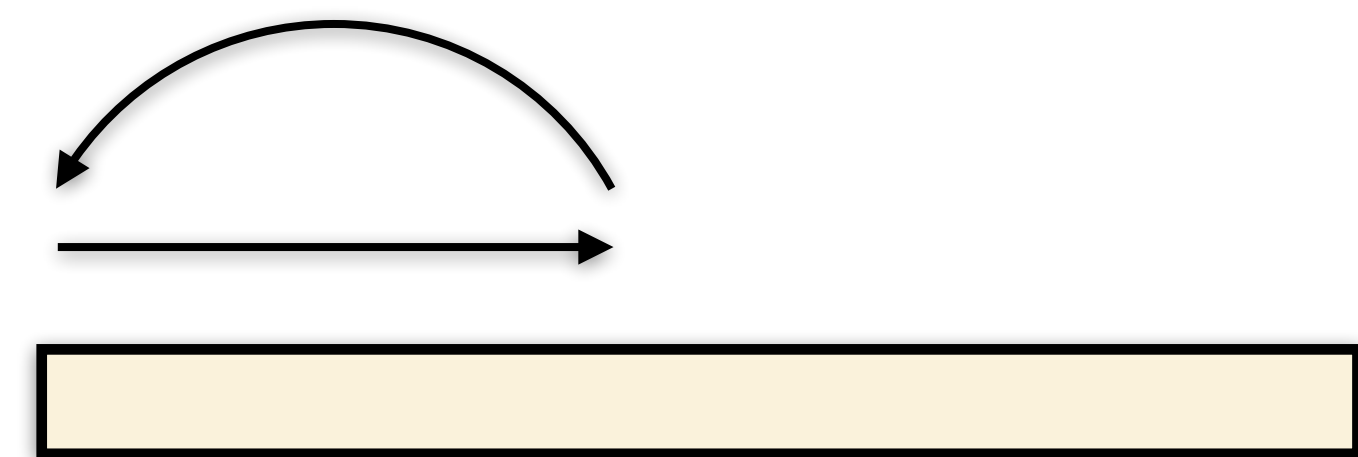
Most programs have a high degree of **locality**.

Spatial locality: how many accesses an algorithm makes to **nearby** data over a short period of time [Denning72, Denning05].



Makes use of **multiple elements transferred together**

Temporal locality: how many repeated accesses an algorithm makes to **the same** data over a short period of time [Denning72, Denning05].



Makes use of **efficient hierarchical accesses**

Memory hierarchy

The **memory hierarchy** takes advantage of locality to speed up the average case to handle memory latency.

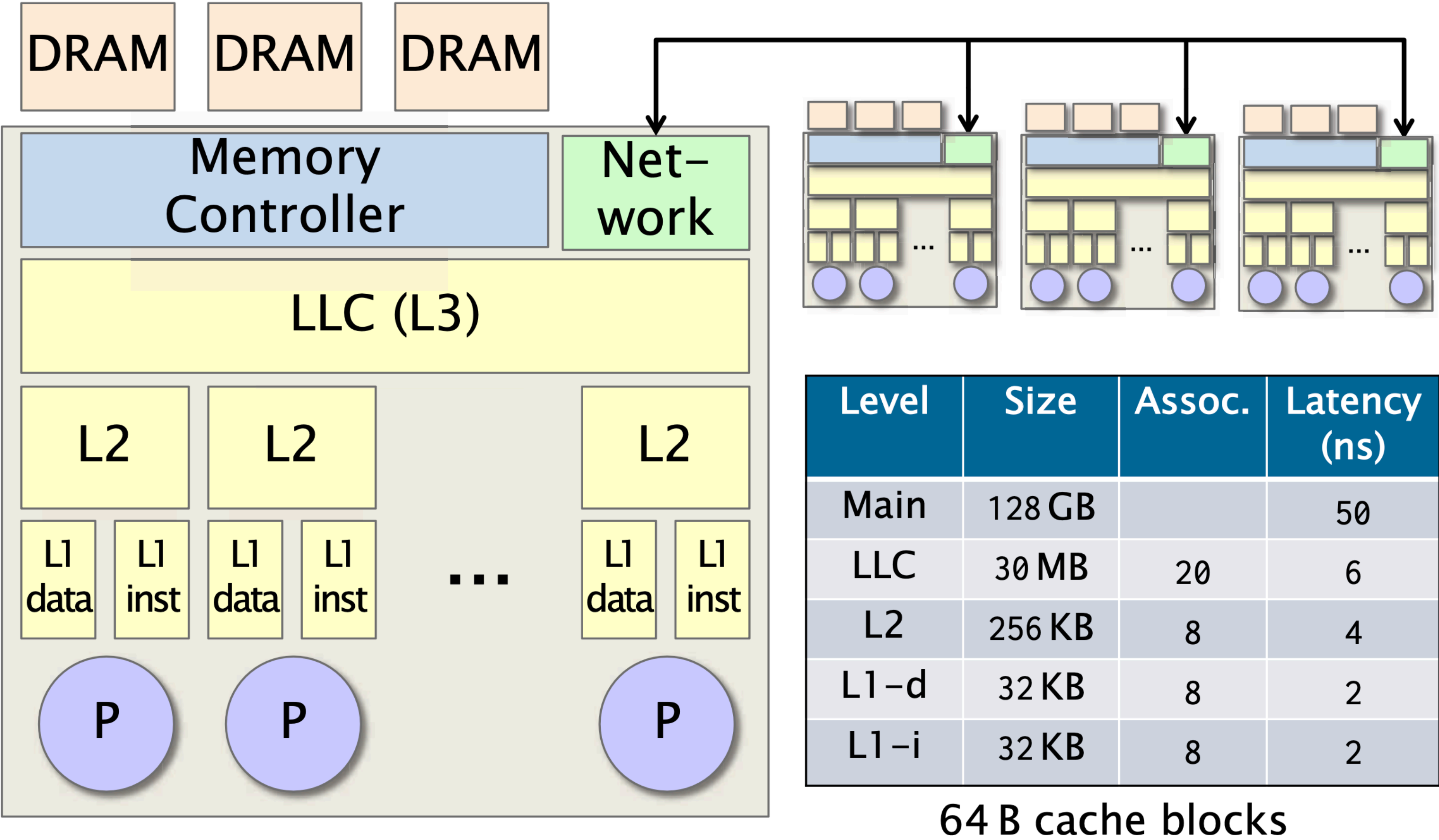


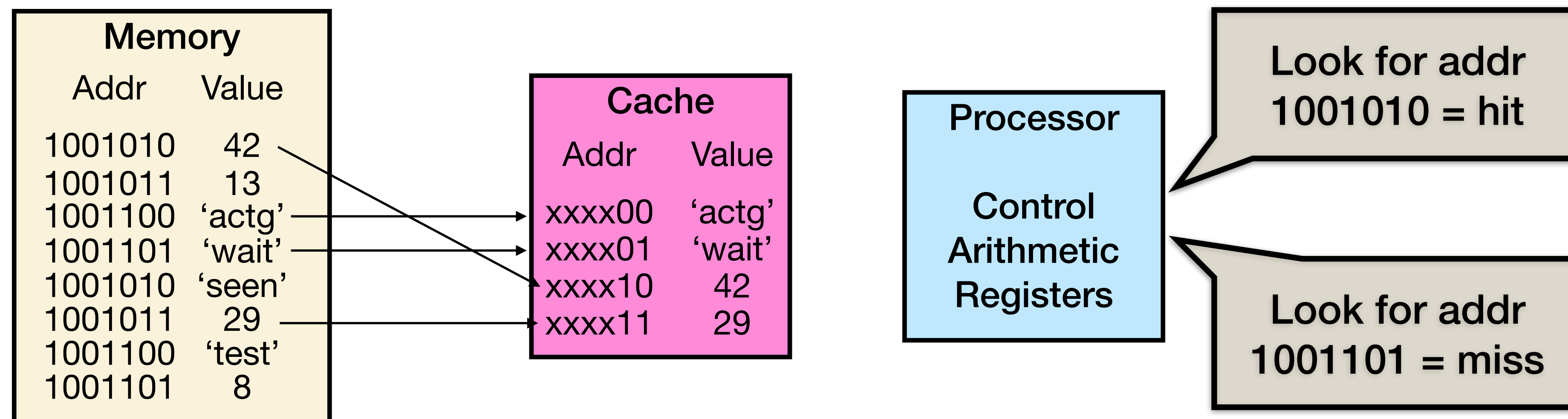
Figure from MIT 6.172

Cache basics

Cache is fast (expensive) memory which keeps a copy of the data; it is hidden from software.

Cache-line length: number of bytes loaded together in one entry (often 64 bytes).

Simple example: data at address xxxxx10 is stored at cache location 10.

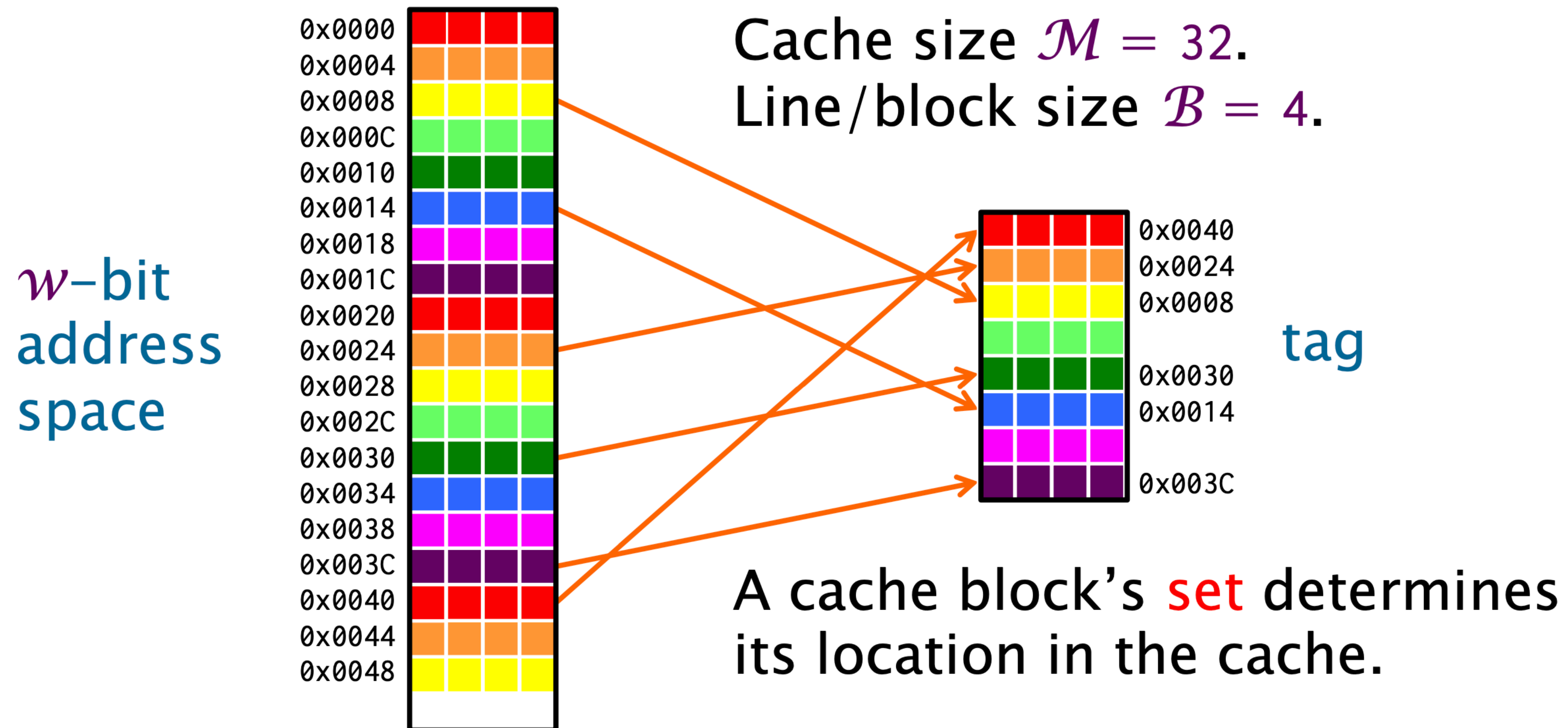


Cache **hit**: access to a memory address in cache - cheap

Cache **miss**: non-cached memory access - expensive

Need to look in next, slower level of memory.

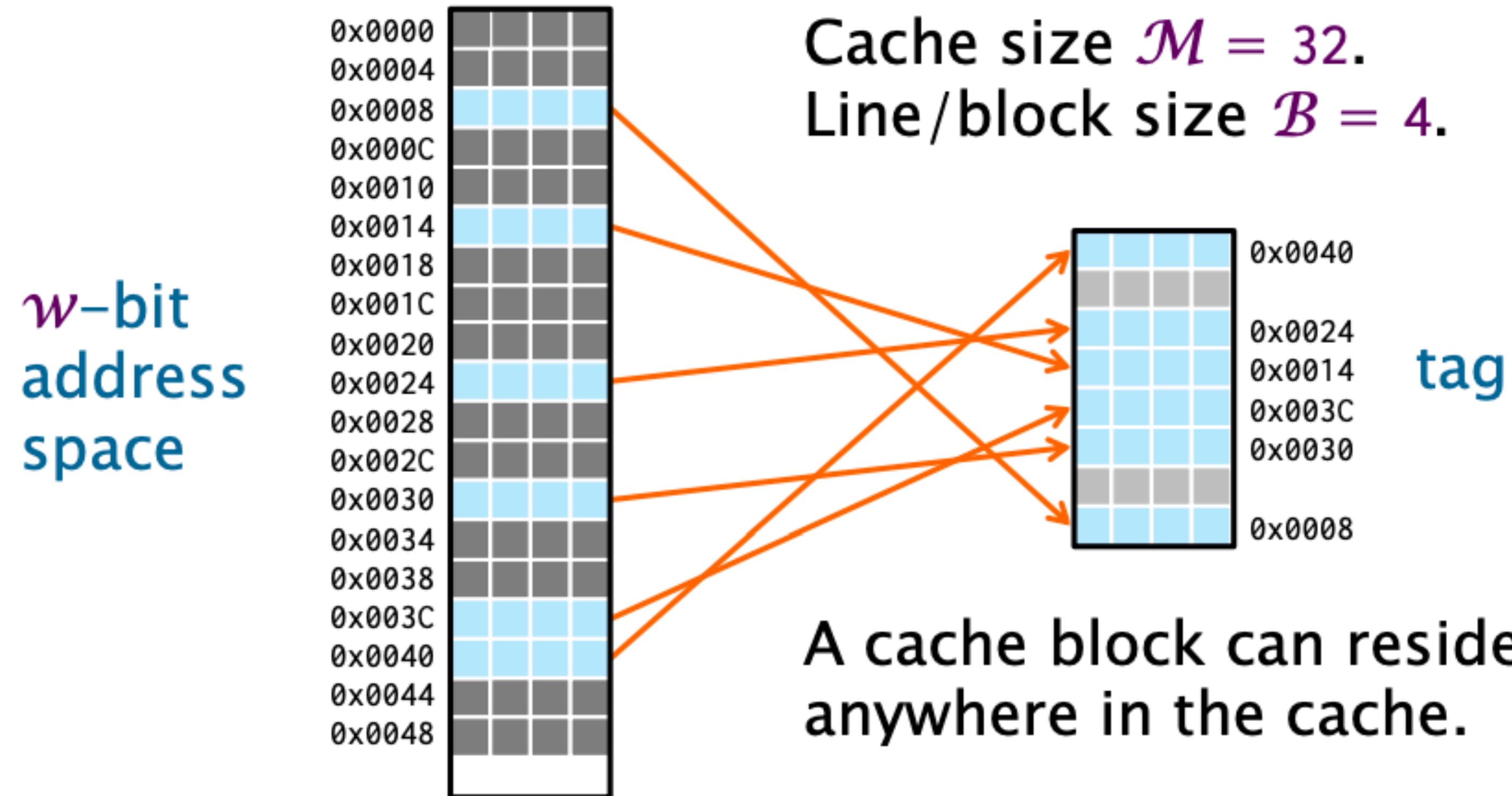
Direct-mapped cache



address		
	tag	set
bits	$w - \lg \mathcal{M}$	$\lg(\mathcal{M}/\mathcal{B})$
	offset	
bits	$\lg \mathcal{B}$	

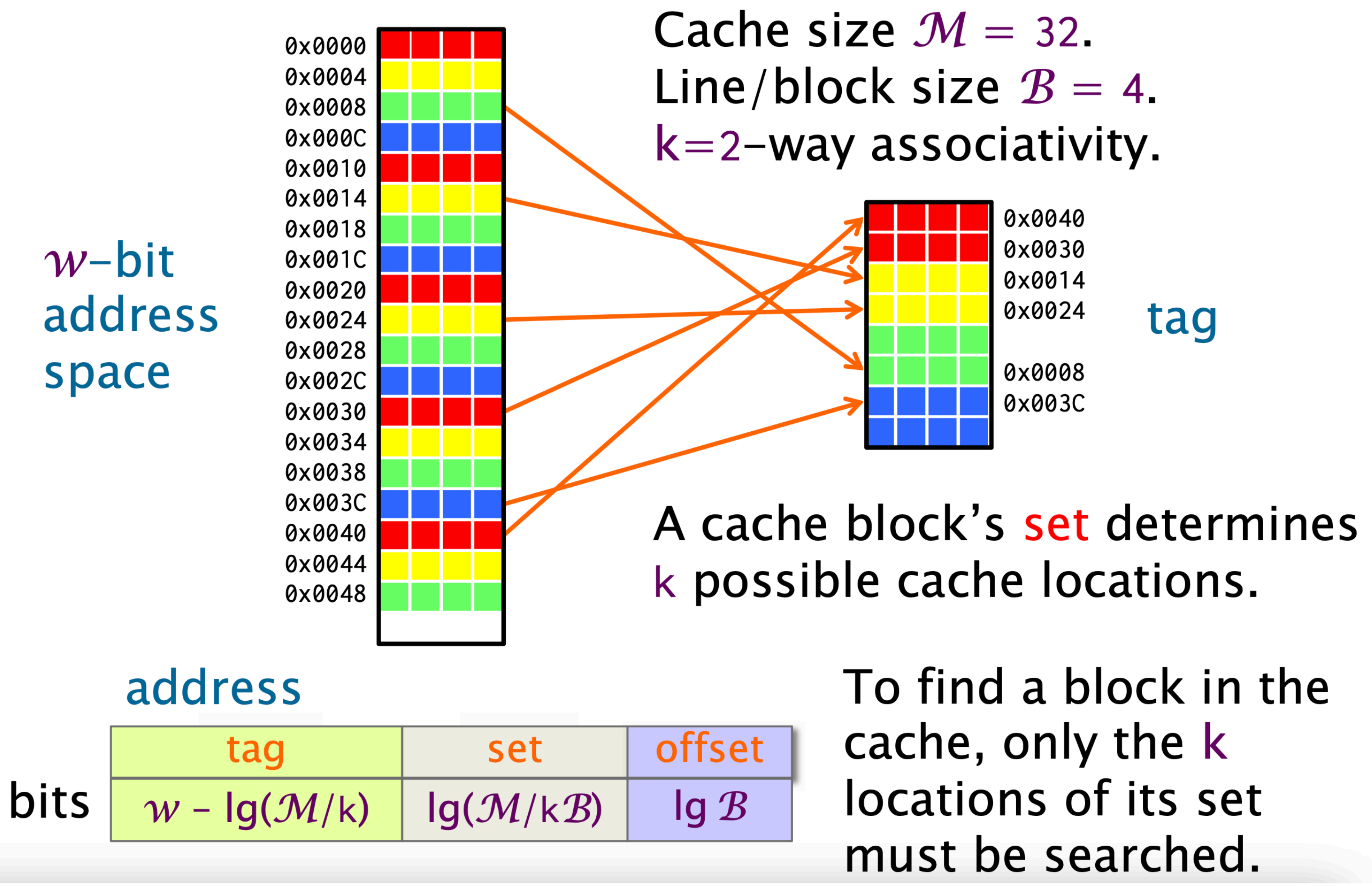
To find a block in the cache, only a single location in the cache need be searched.

Fully-associative cache



To find a block in the cache, the entire cache must be searched for the tag. When the cache becomes full, a block must be **evicted** to make room for a new block. The **replacement policy** determines which block to evict.

Set-associative cache



Type of cache misses - Three C's

Cold miss

The first time a cache block is accessed.

Capacity miss

The previous cached copy would have been evicted even with a fully-associative cache.

Conflict miss

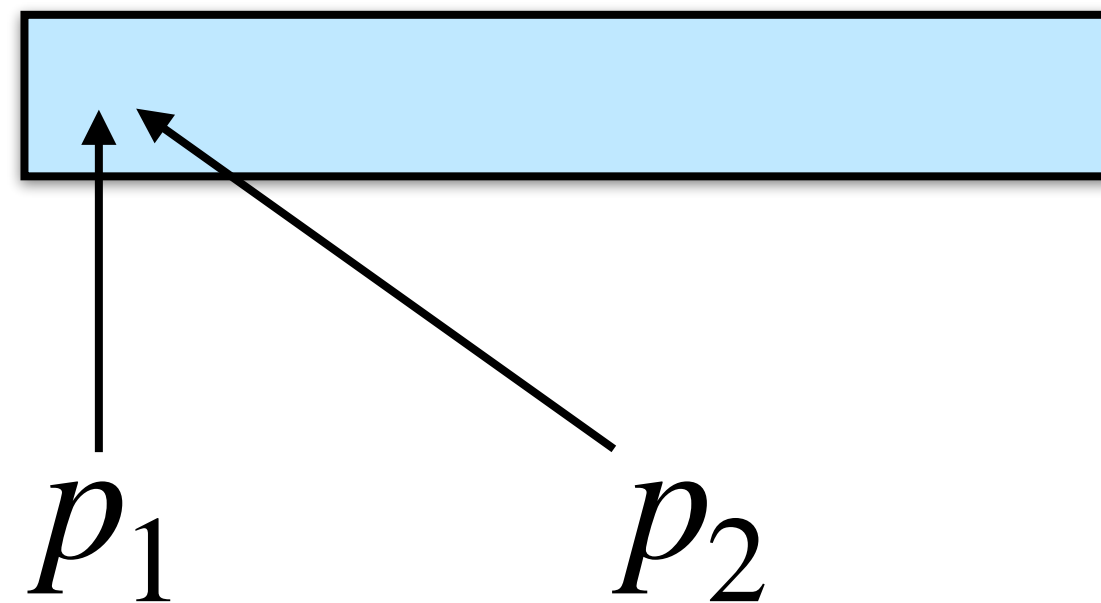
Too many blocks from the same set in cache. The block would not have been evicted with a fully-associative cache.

Sharing misses

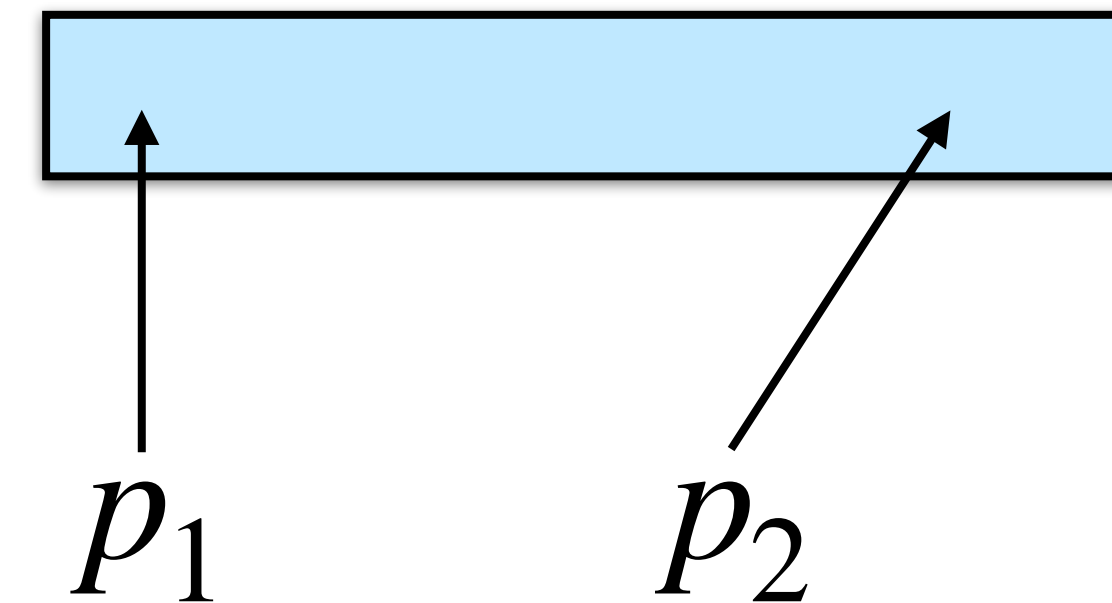
Sharing miss

- Another processor acquired exclusive access to the cache block.
- True-sharing miss: The two processors are accessing the same data on the cache line.
- False-sharing miss: The two processors are accessing different data that happen to reside on the same cache line.

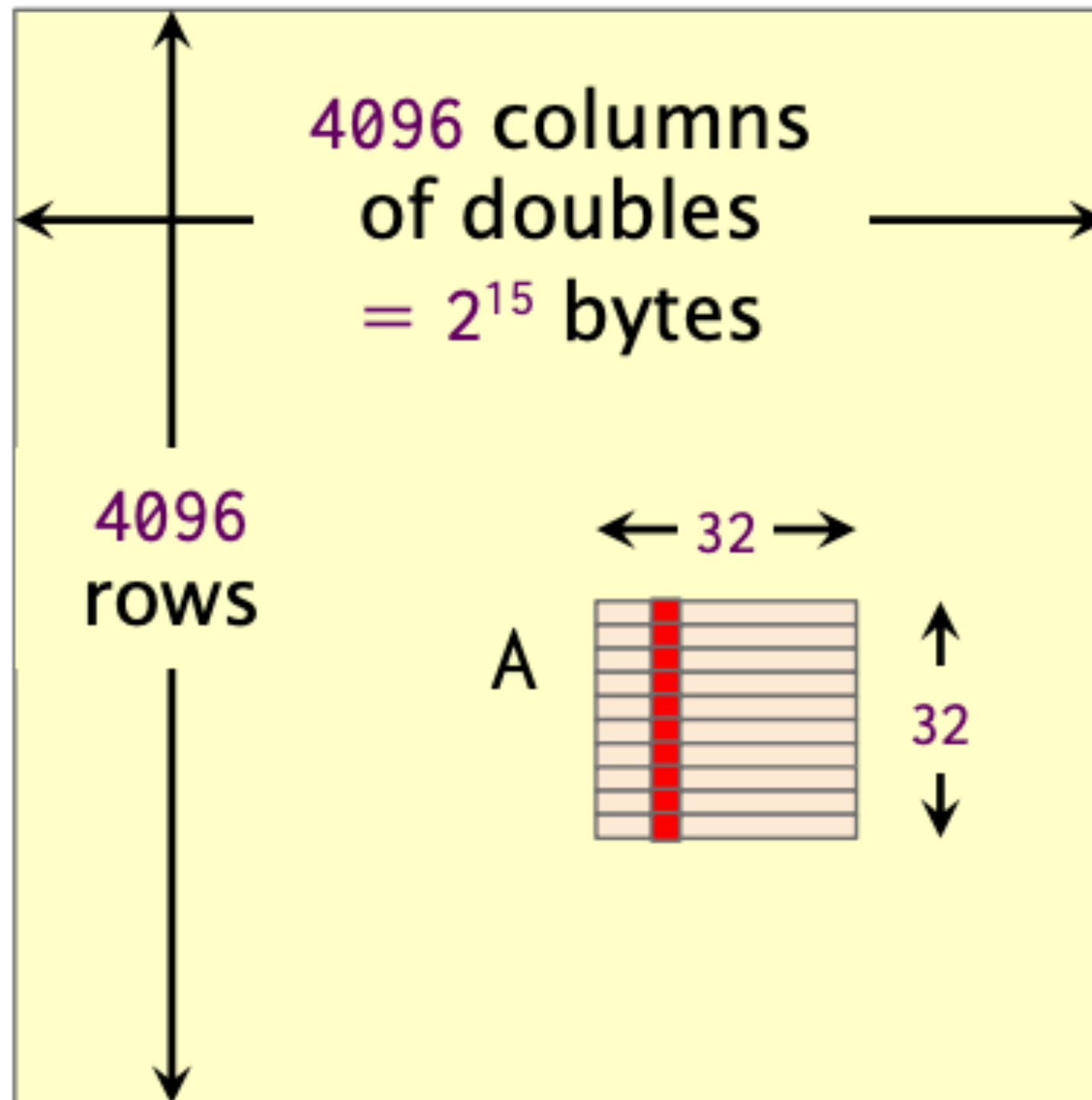
True Sharing



False Sharing



Conflict misses for submatrices



Assume:

- Word width $w = 64$.
- Cache size $\mathcal{M} = 32K$.
- Line (block) size $\mathcal{B} = 64$.
- $k=4$ -way associativity.

Conflict misses can be problematic for caches with limited associativity.

address		
tag	set	offset
$w - \lg(\mathcal{M}/k)$	$\lg(\mathcal{M}/k\mathcal{B})$	$\lg \mathcal{B}$
51	7	6

bits

Analysis

Look at a column of submatrix A . The addresses of the elements are $x, x+2^{15}, x+2 \cdot 2^{15}, \dots, x+31 \cdot 2^{15}$.

They all fall into the same set!

Solutions

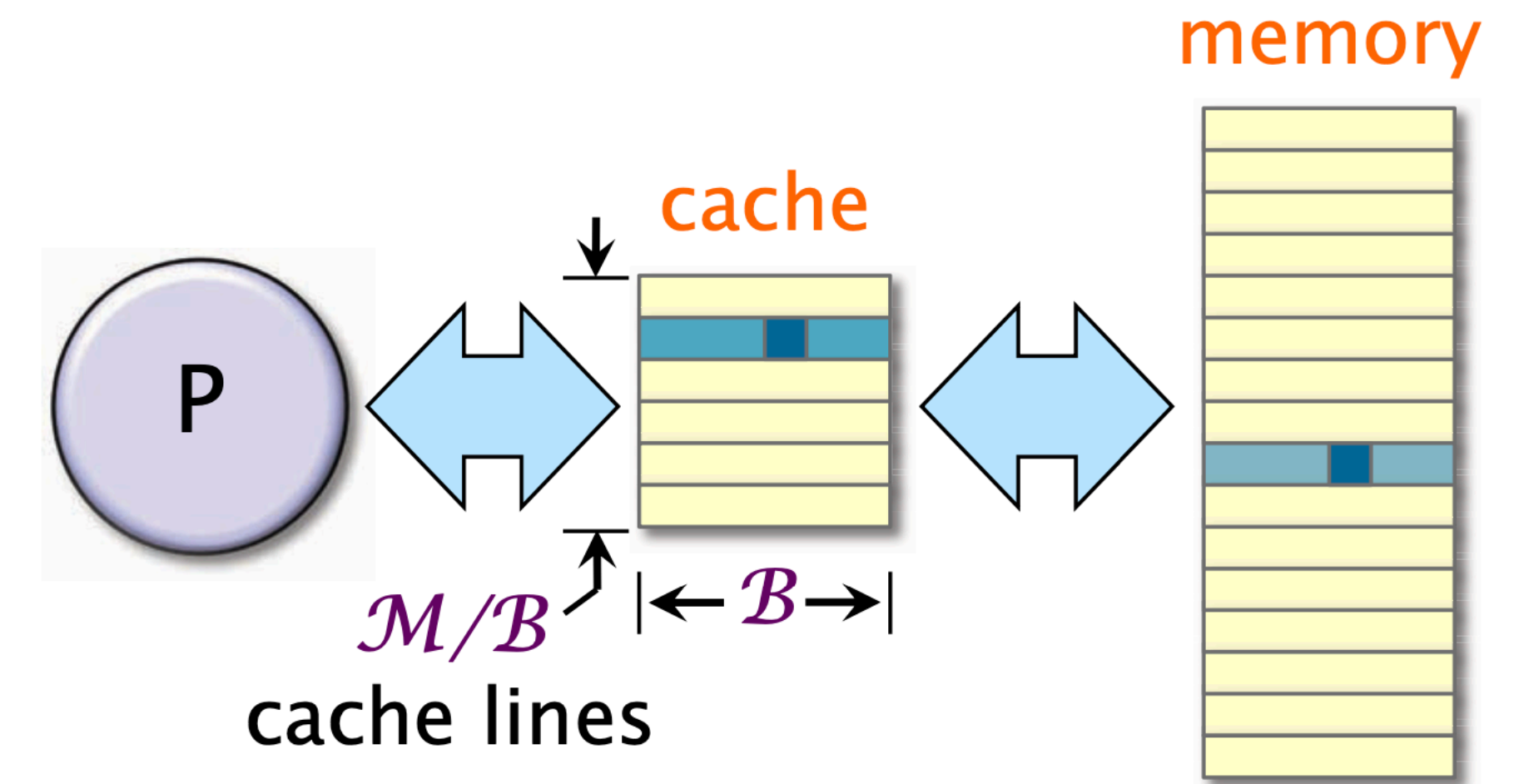
Copy A into a temporary 32×32 matrix, or pad rows.

Ideal-Cache Model

Ideal-Cache Model

Parameters

- Two-level hierarchy
- Cache size of $\mathcal{M}$ bytes
- Cache-line length of $\mathcal{B}$ bytes
- Fully associative
- Optimal, omniscient replacement.



Performance Measures

- **Work** W (ordinary running time)
- **Cache misses** Q (number of cache lines that need to be transferred between cache and memory)

How reasonable are ideal caches?

“LRU” Lemma [ST85]. Suppose that an algorithm incurs Q cache misses on an ideal cache of size $\mathcal{M}$. Then on a fully associative cache of size $2\mathcal{M}$ that uses the **least-recently used (LRU)** replacement policy, it incurs at most $2Q$ cache misses.

Implication

For asymptotic analyses, one can assume optimal or LRU replacement, as convenient.

Software engineering

- Design a **theoretically good** algorithm.
- **Engineer** for detailed performance.
- Real caches are not fully associative.
- Loads and stores have different costs with respect to bandwidth and latency.

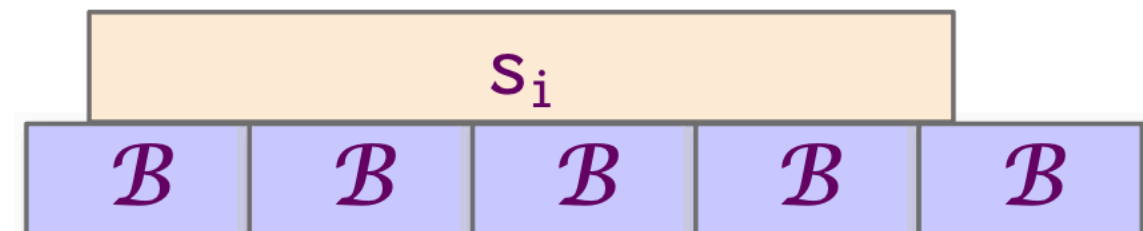
Cache-miss lemma

Lemma. Suppose that a program reads a set of r data segments, where the i th segment consists of s_i bytes, and suppose that

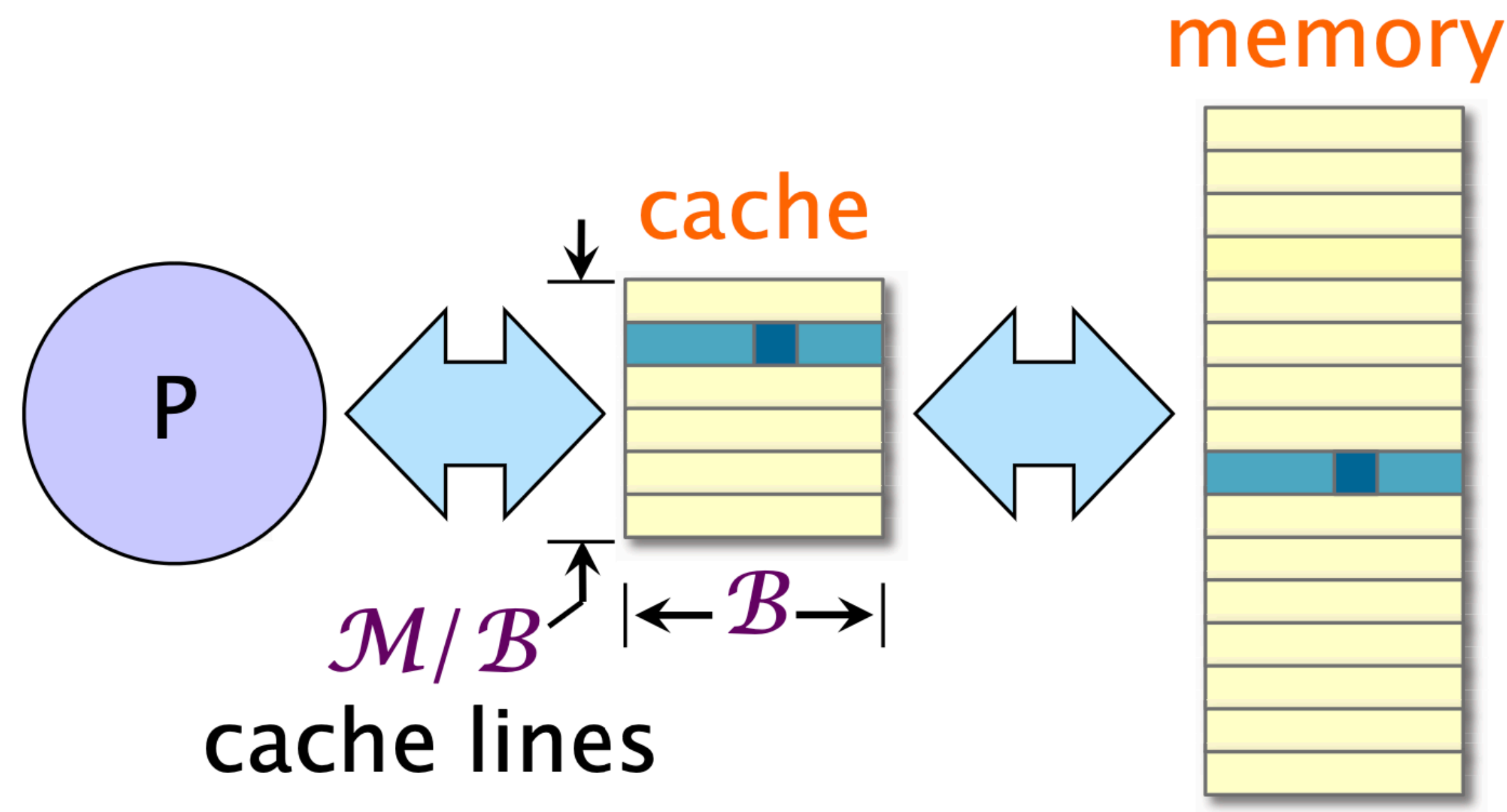
$$\sum_{i=1}^r s_i = N < \mathcal{M}/3 \text{ and } N/r \geq \mathcal{B}.$$

Then all of the segments fit into cache, and the number of misses to read them all is at most $3N/\mathcal{B}$.

Proof. Suppose that a program reads a set of r data segments, where the i th segment consists of A single segment s_i incurs at most $s_i/\mathcal{B}$ misses, and so

$$\begin{aligned} \sum_{i=1}^r (s_i/\mathcal{B} + 2) &= N/\mathcal{B} + 2r \\ &= N/\mathcal{B} + (2r\mathcal{B})/\mathcal{B} \\ &\leq N/\mathcal{B} + 2N/\mathcal{B} \\ &= 3N/\mathcal{B}. \quad \blacksquare \end{aligned}$$


Tall caches



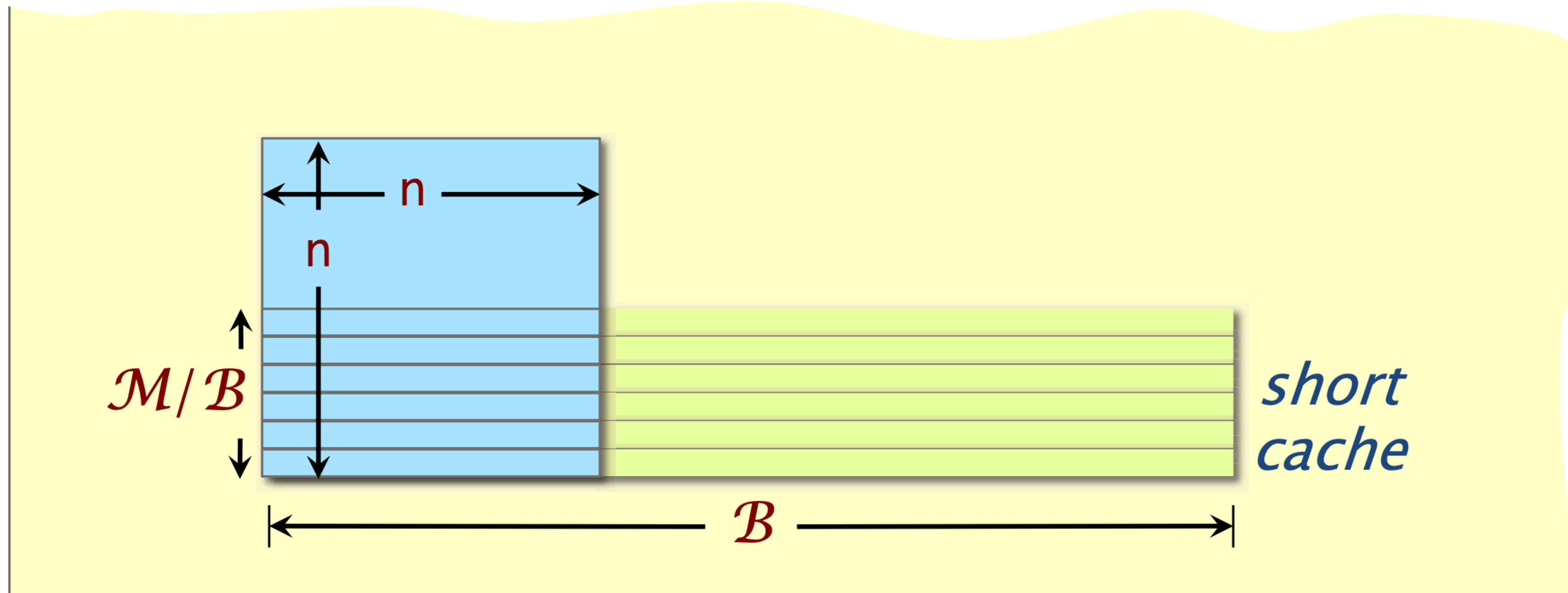
Tall-cache assumption

$\mathcal{B}^2 < c \mathcal{M}$ for some sufficiently small constant $c \leq 1$.

Example: Intel Xeon E5-2666 v3

- Cache-line length = 64 bytes.
- L1-cache size = 32 Kbytes.

What's wrong with short caches?

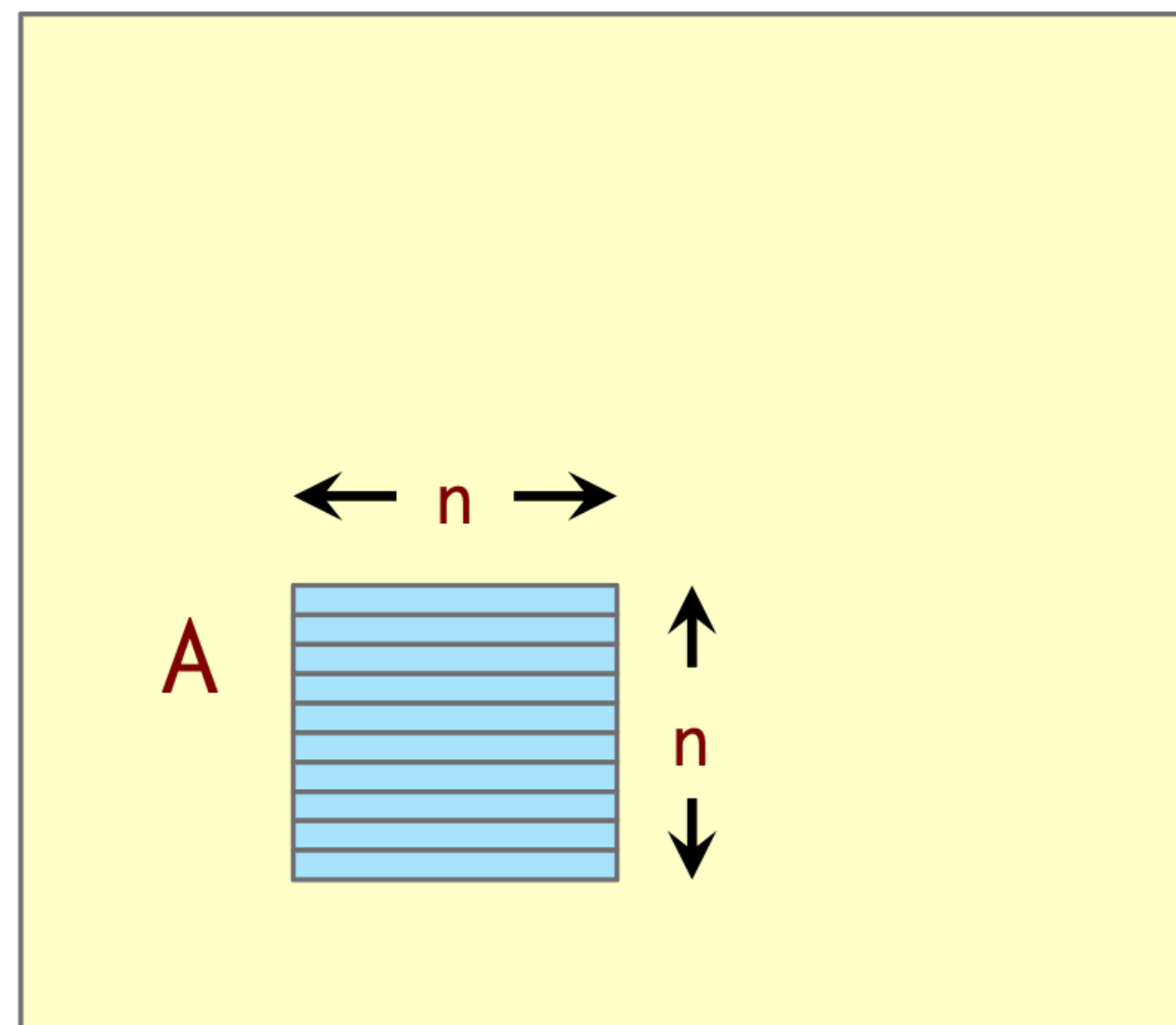


Tall-cache assumption

$B^2 < c\mathcal{M}$ for some sufficiently small constant $c \leq 1$.

An $n \times n$ submatrix stored in row-major order may not fit in a short cache even if $n^2 < c\mathcal{M}$!

Submatrix caching lemma



Lemma. Suppose that an $n \times n$ submatrix A is read into a tall cache satisfying $\mathcal{B}^2 < c\mathcal{M}$, where $c \leq 1$ is constant, and suppose that $c\mathcal{M} \leq n^2 < \mathcal{M}/3$. Then A fits into cache, and the number of misses to read all A 's elements is at most $3n^2/\mathcal{B}$.

Proof. We have $N = n^2$, $n = r = s_i$, $\mathcal{B} \leq n = N/r$, and $N < \mathcal{M}/3$. Thus, the Cache-Miss Lemma applies. ■

Cache Analysis of Matrix Multiplication

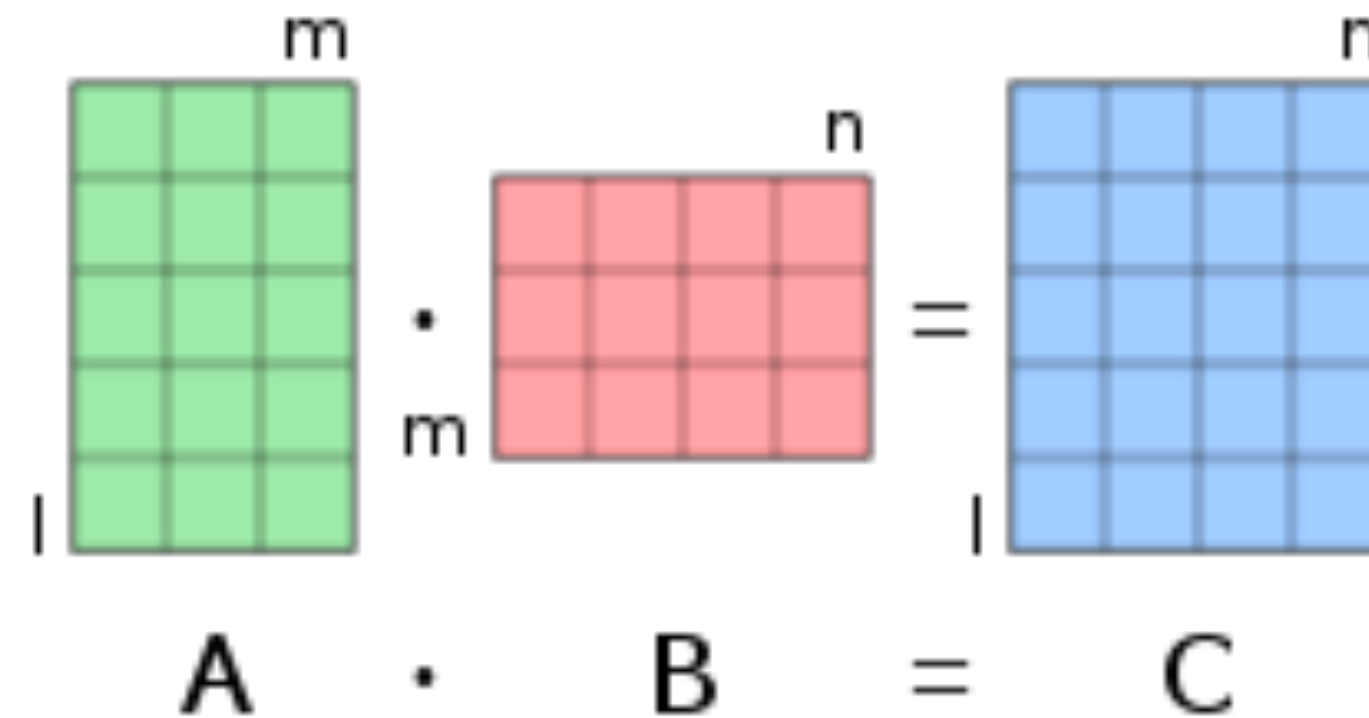
Why matrix multiplication?

Matrix multiplication is an **important kernel** in many problems:

- Dense linear algebra is a motif in every list,
- Closely related to other algorithms, e.g., transitive closure on a graph,
- And dominates training time in deep learning (CNNs)

Good model problem (well-studied, illustrates ideas).

Easy to find good libraries that are hard to beat! (e.g., Intel MKL, etc.)



Multiply square matrices

```
void Mult(double *C, double *A, double *B, int64_t n) {  
    for (int64_t i=0; i < n; i++)  
        for (int64_t j=0; j < n; j++)  
            for (int64_t k=0; k < n; k++)  
                C[i*n+j] += A[i*n+k] * B[k*n+j];  
}
```

Analysis of work:

$$W(n) = \Theta(n^3)$$

Memory layout of matrices

In this matrix-multiplication code, matrices are laid out in memory in **row-major order**.

Matrix

Row 1
Row 2
Row 3
Row 4
Row 5
Row 6
Row 7
Row 8

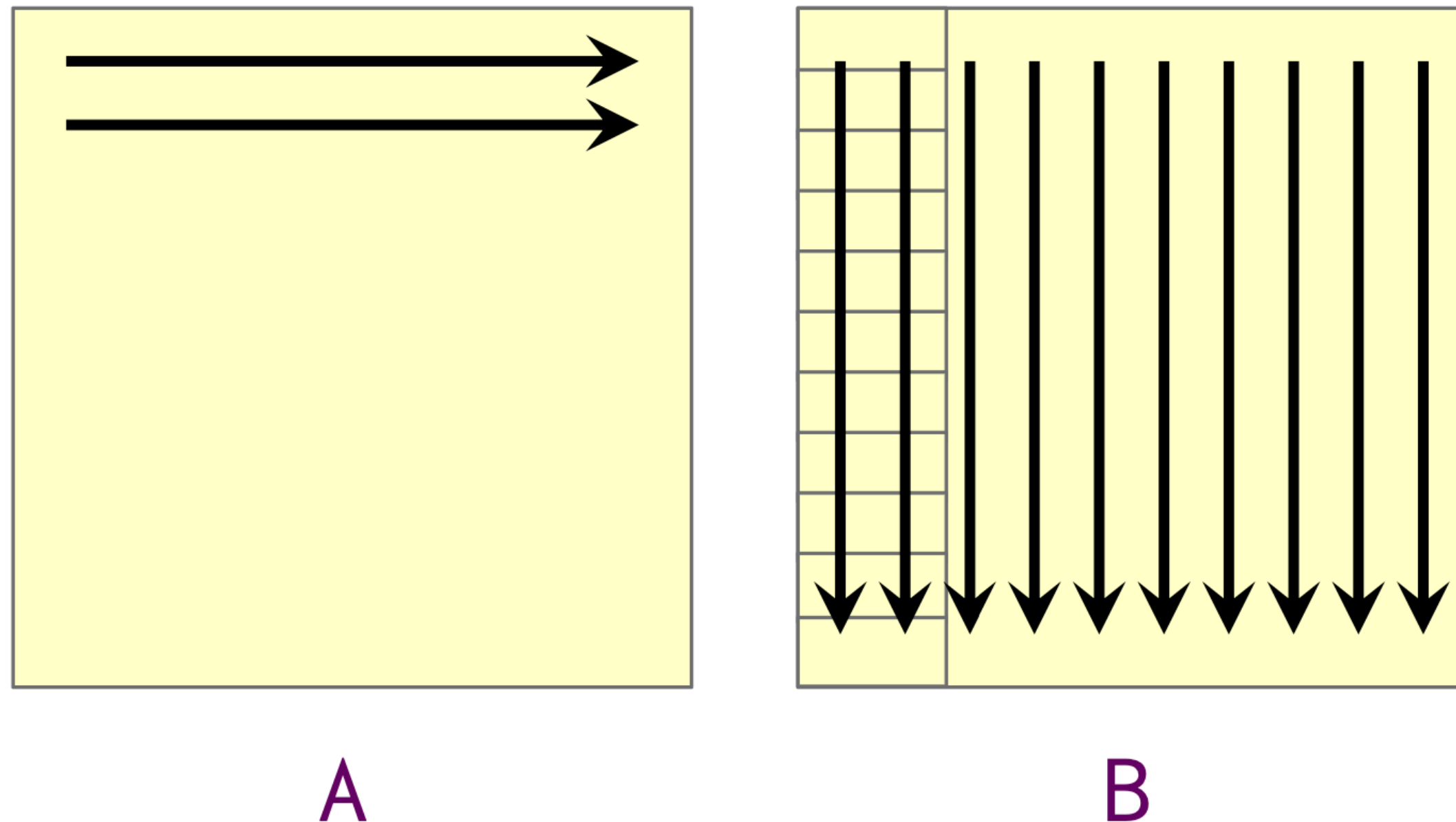
Memory

Row 1	Row 2	Row 3
-------	-------	-------

Analysis of cache misses

```
void Mult(double *C, double *A, double *B, int64_t n) {  
    for (int64_t i=0; i < n; i++)  
        for (int64_t j=0; j < n; j++)  
            for (int64_t k=0; k < n; k++)  
                C[i*n+j] += A[i*n+k] * B[k*n+j];  
}
```

Assume row major and tall cache



Case 1

$n > c\mathcal{M}/\mathcal{B}$. Analyze matrix **B**.

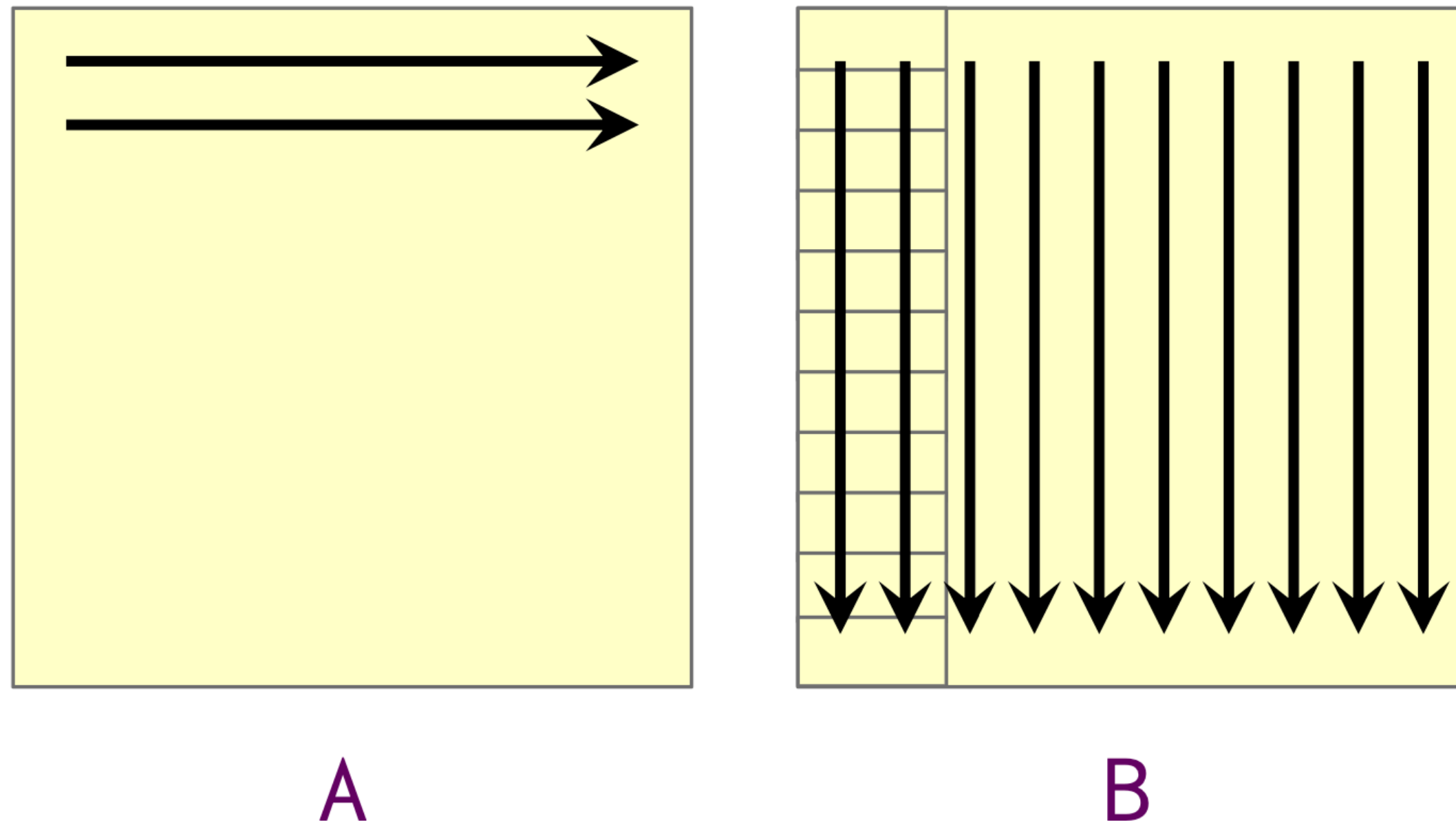
Assume LRU.

$Q(n) = \Theta(n^3)$, since matrix **B** misses on every access.

Analysis of cache misses

```
void Mult(double *C, double *A, double *B, int64_t n) {  
    for (int64_t i=0; i < n; i++)  
        for (int64_t j=0; j < n; j++)  
            for (int64_t k=0; k < n; k++)  
                C[i*n+j] += A[i*n+k] * B[k*n+j];  
}
```

Assume row major and tall cache



Case 2

$c' \mathcal{M}^{1/2} < n < c \mathcal{M} / \mathcal{B}$. Analyze matrix **B**.

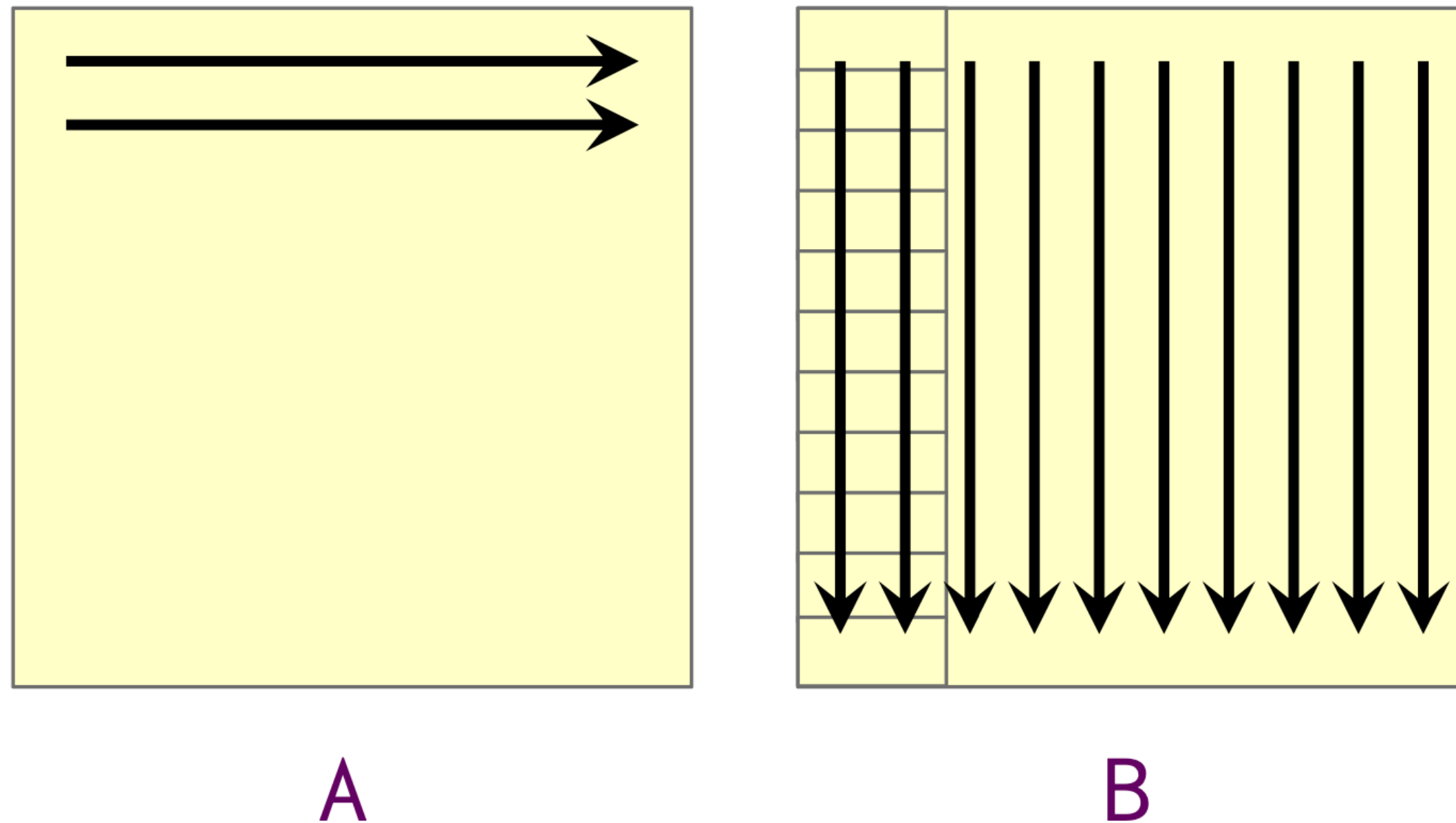
Assume LRU.

$Q(n) = n \cdot \Theta(n^2 / \mathcal{B}) = \Theta(n^3 / \mathcal{B})$,
since matrix **B** can exploit spatial locality.

Analysis of cache misses

```
void Mult(double *C, double *A, double *B, int64_t n) {  
    for (int64_t i=0; i < n; i++)  
        for (int64_t j=0; j < n; j++)  
            for (int64_t k=0; k < n; k++)  
                C[i*n+j] += A[i*n+k] * B[k*n+j];  
}
```

Assume row major and tall cache



Case 3

$n < c' \mathcal{M}^{1/2}$. Analyze matrix **B**.

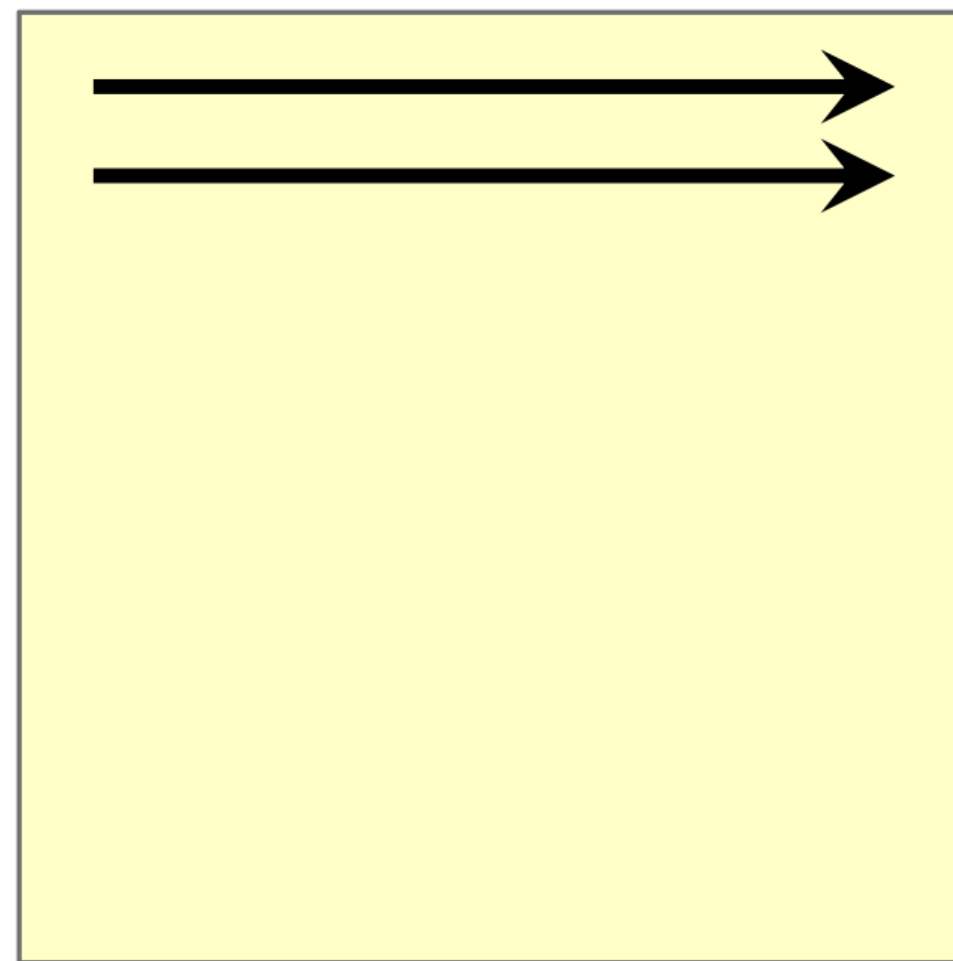
Assume LRU.

$Q(n) = \Theta(n^2 / \mathcal{B})$, since everything fits in cache!

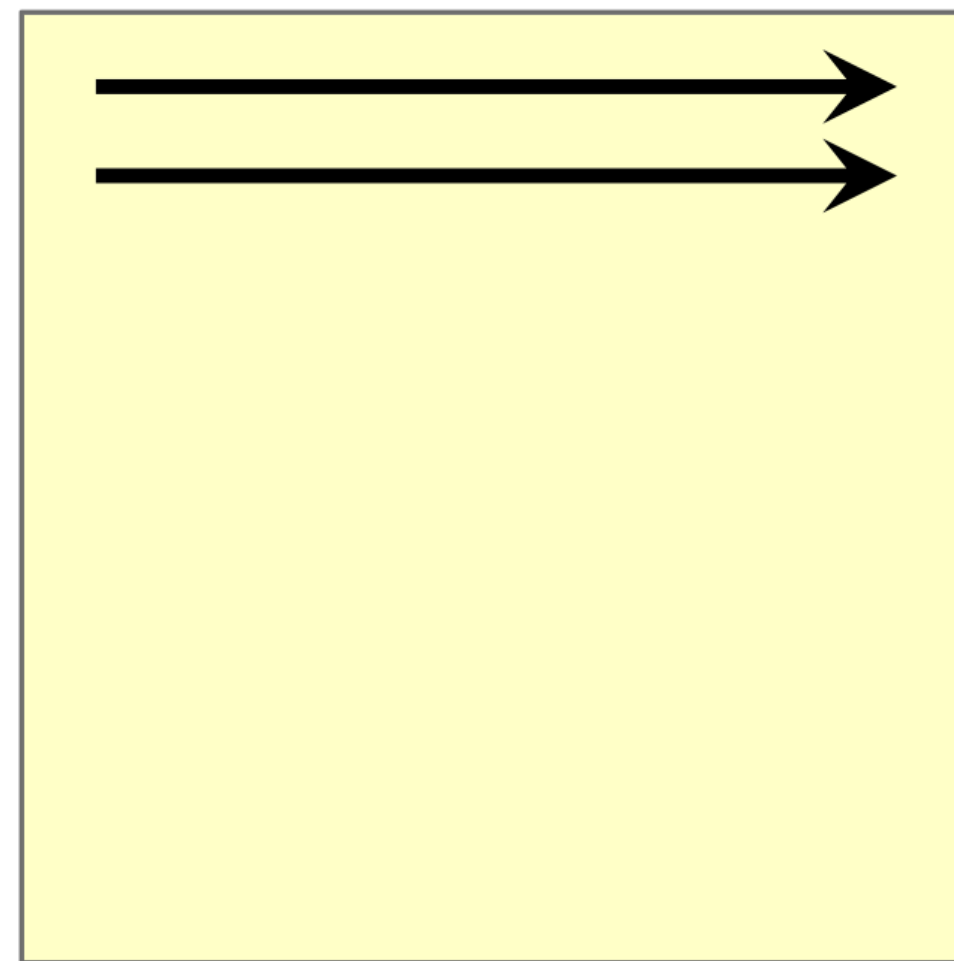
Swapping inner loop order

```
void Mult(double *C, double *A, double *B, int64_t n) {  
    for (int64_t i=0; i < n; I++)  
        for (int64_t k=0; k < n; k++)  
            for (int64_t j=0; j < n; j++)  
                C[i*n+j] += A[i*n+k] * B[k*n+j];  
}
```

Assume row major and tall cache



C



B

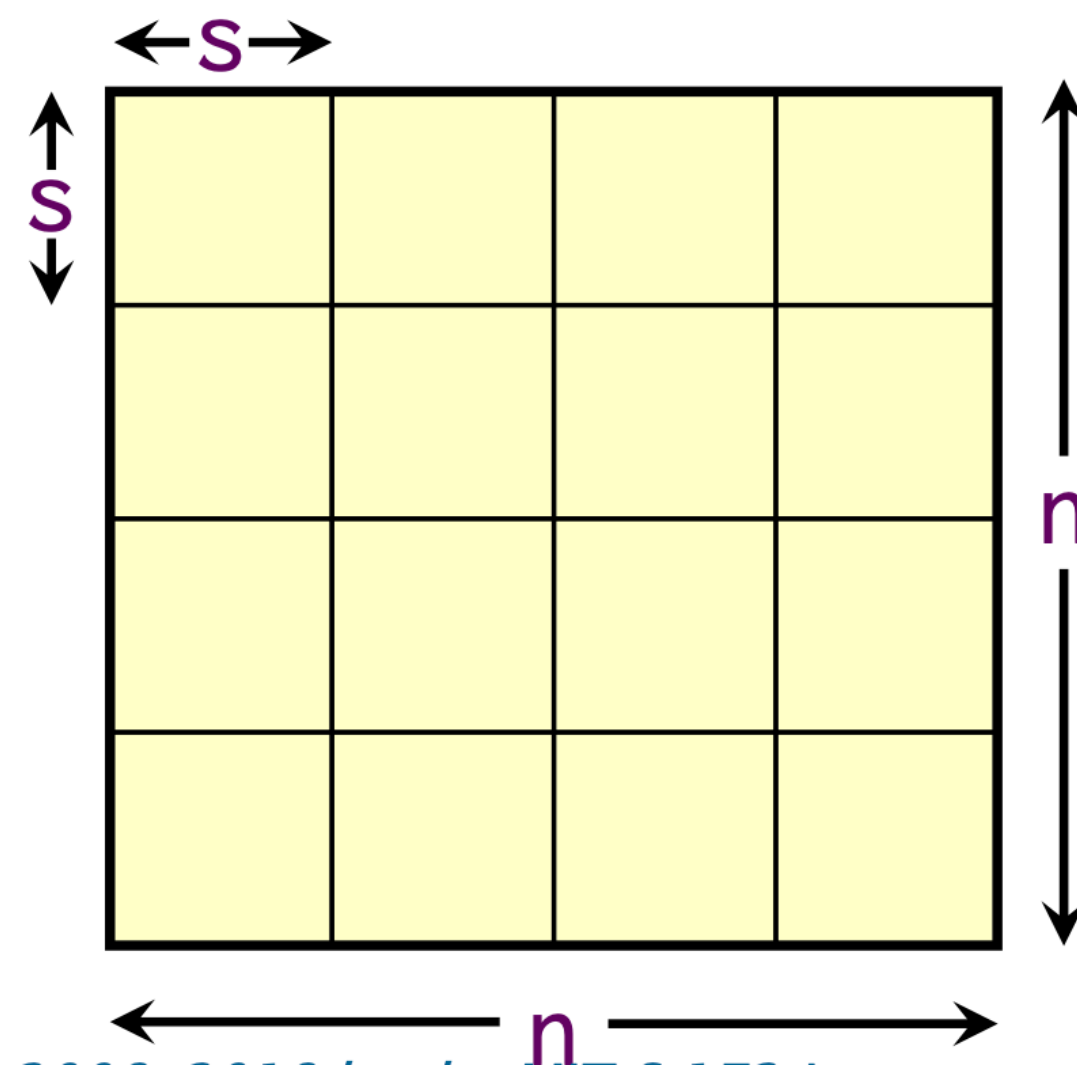
Assume matrix B. Assume LRU.

$Q(n) = n \cdot \Theta(n^2/\mathcal{B}) = \Theta(n^3/\mathcal{B})$,
since matrix **B** can exploit spatial
locality.

Tiling (aka Blocking)

Tiled (Blocked) matrix multiply

```
void Tiled_Mult(double *C, double *A, double *B, int64_t n) {  
    for (int64_t i1=0; i1<n; i1+=s)  
        for (int64_t j1=0; j1<n; j1+=s)  
            for (int64_t k1=0; k1<n; k1+=s)  
                for (int64_t i=i1; i<i1+s && i<n; i++)  
                    for (int64_t j=j1; j<j1+s && j<n; j++)  
                        for (int64_t k=k1; k<k1+s && k<n; k++)  
                            C[i*n+j] += A[i*n+k] * B[k*n+j];  
}
```



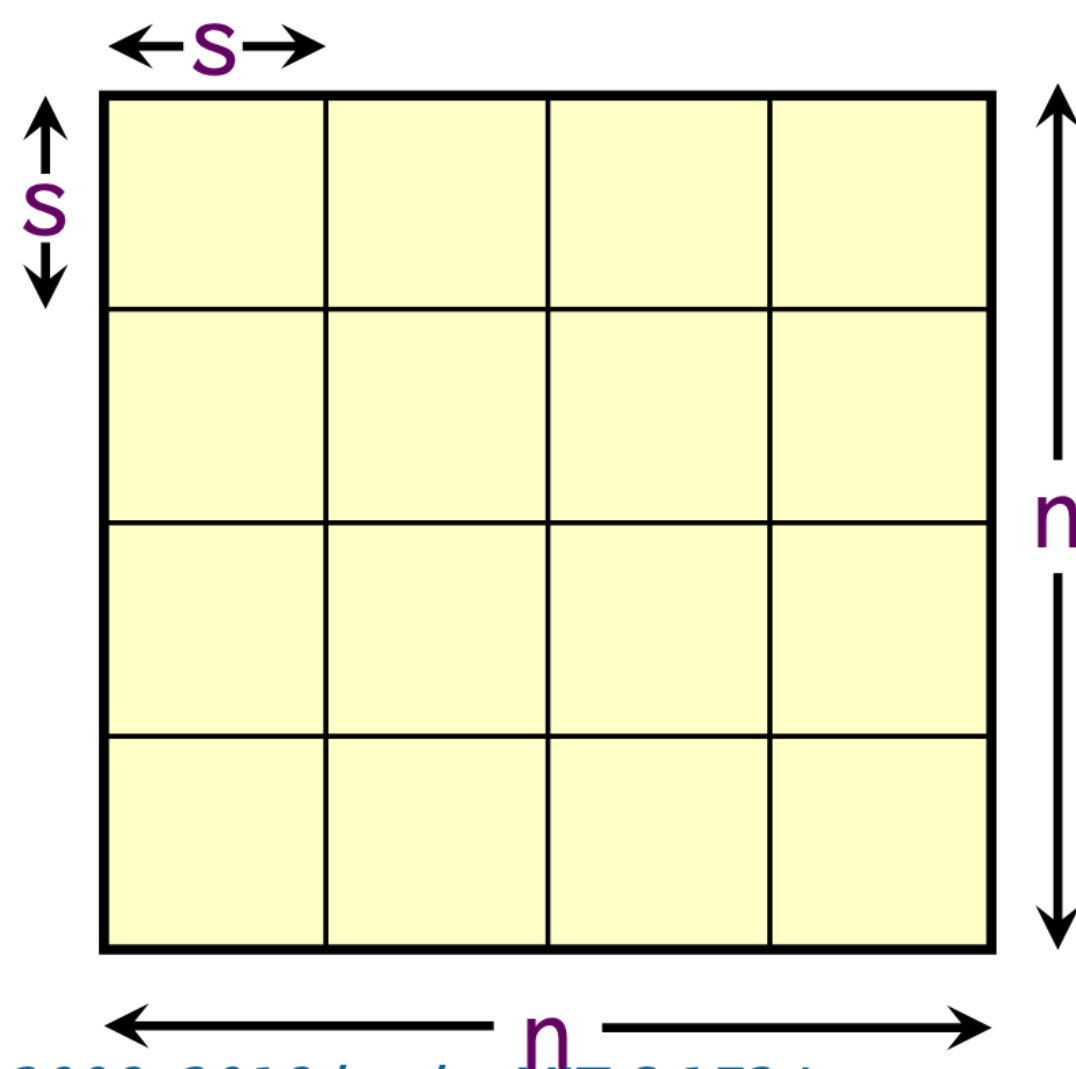
Analysis of work:

$$\begin{aligned} W(n) &= \Theta((n/s)^3 (s)^3) \\ &= \Theta(n^3) \end{aligned}$$

Tile size (or
block size)

Tiled (Blocked) matrix multiply

```
void Tiled_Mult(double *C, double *A, double *B, int64_t n) {  
    for (int64_t i1=0; i1<n; i1+=s)  
        for (int64_t j1=0; j1<n; j1+=s)  
            for (int64_t k1=0; k1<n; k1+=s)  
                for (int64_t i=i1; i<i1+s && i<n; i++)  
                    for (int64_t j=j1; j<j1+s && j<n; j++)  
                        for (int64_t k=k1; k<k1+s && k<n; k++)  
                            C[i*n+j] += A[i*n+k] * B[k*n+j];  
}
```



Analysis of cache misses

Tune s so that the submatrices just fit into cache:

$$s = \Theta(\mathcal{M}^{1/2}).$$

Submatrix Caching Lemma implies $\Theta(s^2/\mathcal{B})$ misses per submatrix.

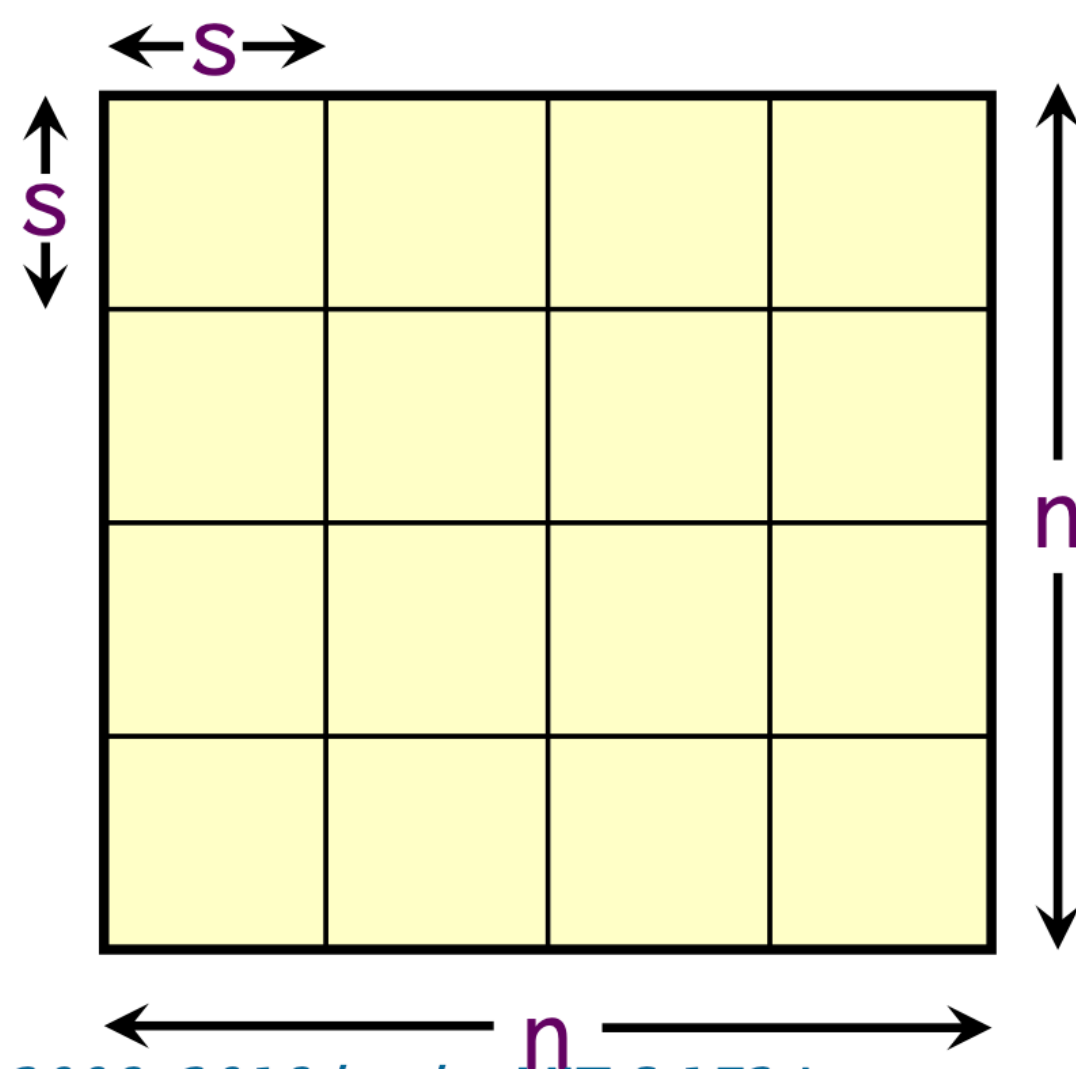
$$Q(n) = \Theta((n/s)^3(s^2/\mathcal{B})) = \Theta(n^3/(\mathcal{B}\mathcal{M}^{1/2})).$$

Optimal
[HK81]

Tiled (Blocked) matrix multiply

```
void Tiled_Mult(double *C, double *A, double *B, int64_t n) {  
    for (int64_t i1=0; i1<n; i1+=s)  
        for (int64_t j1=0; j1<n; j1+=s)  
            for (int64_t k1=0; k1<n; k1+=s)  
                for (int64_t i=i1; i<n; i++)  
                    for (int64_t j=j1; j<n; j++)  
                        for (int64_t k=k1; k<n; k++)  
                            C[i*n+j] += A[i*n+k] * B[k*n+j];  
}
```

How?



Analysis of cache misses

Tune s so that the submatrices just fit into cache:

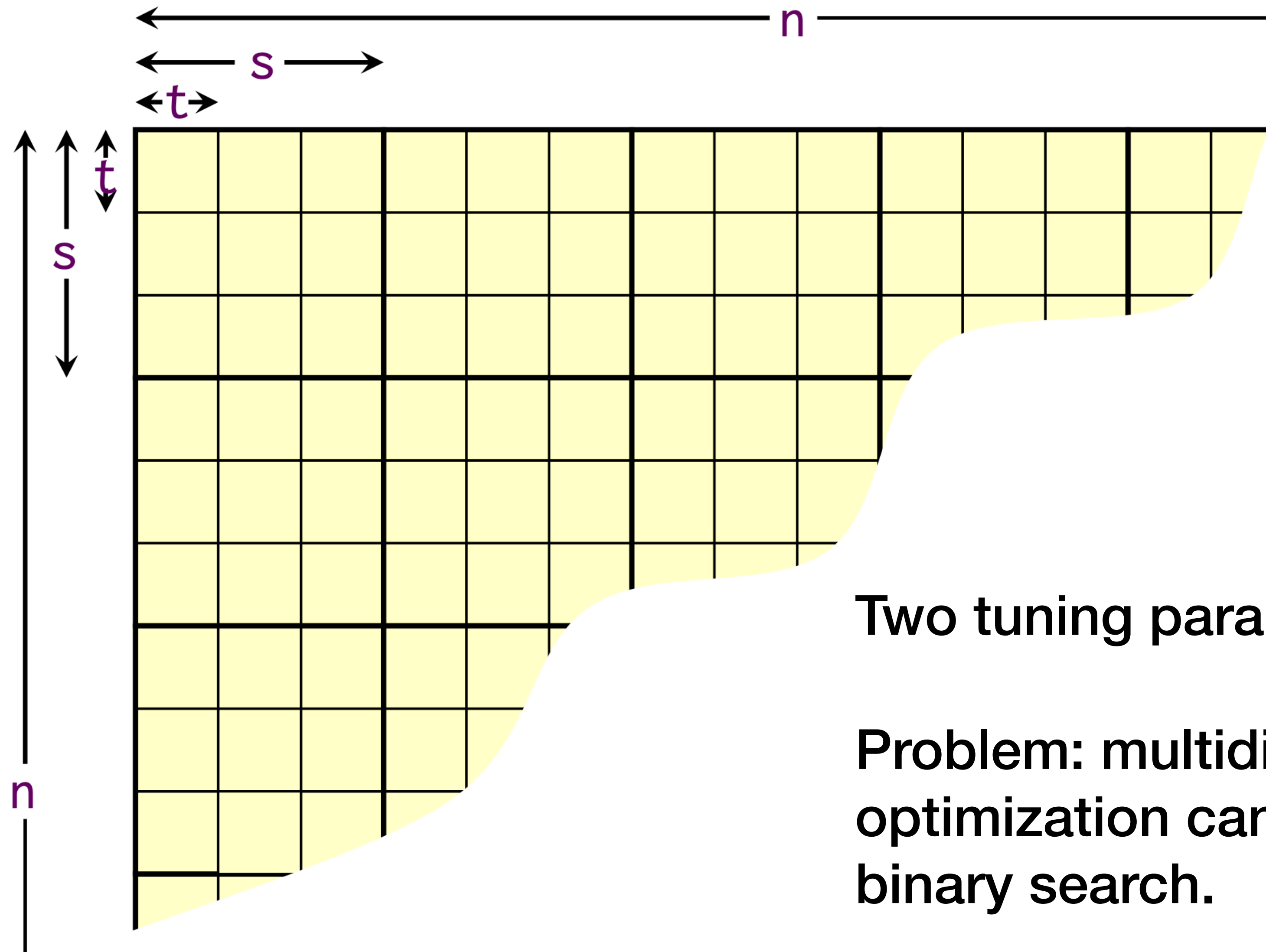
$$s = \Theta(\mathcal{M}^{1/2}).$$

Submatrix Caching Lemma implies $\Theta(s^2/\mathcal{B})$ misses per submatrix.

$$Q(n) = \Theta((n/s)^3(s^2/\mathcal{B})) = \Theta(n^3/(\mathcal{B}\mathcal{M}^{1/2})).$$

Optimal
[HK81]

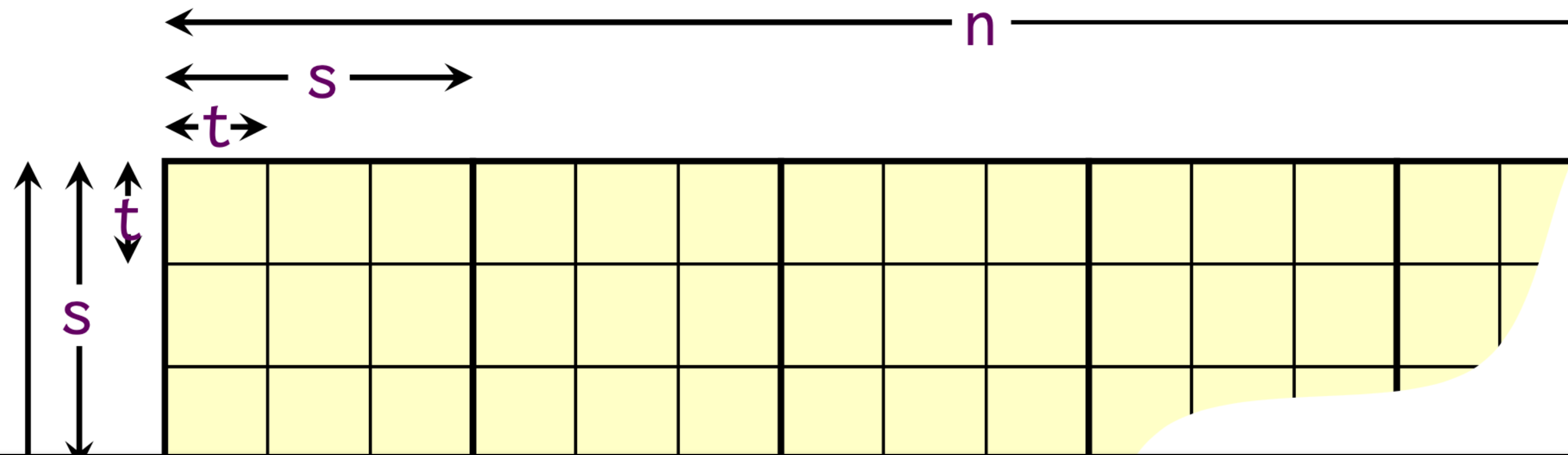
Two-level cache



Two tuning parameters: s and t .

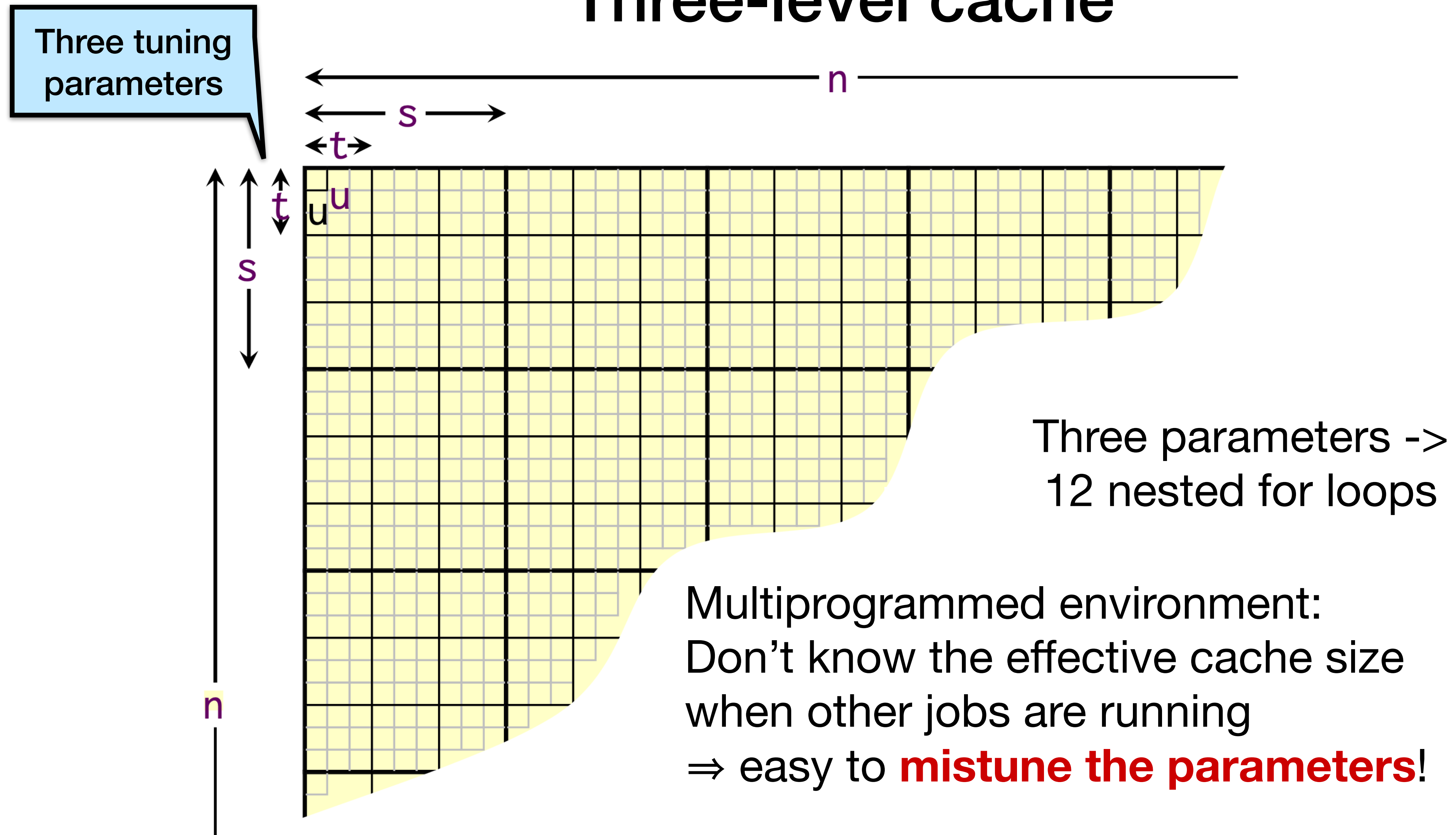
Problem: multidimensional tuning optimization cannot be done with binary search.

Two-level cache



```
void Tiled_Mult2(double *C, double *A, double *B, int64_t n) {  
    for (int64_t i2=0; i2<n; i2+=s)  
        for (int64_t j2=0; j2<n; j2+=s)  
            for (int64_t k2=0; k2<n; k2+=s)  
                for (int64_t i1=i2; i1<i2+s && i1<n; i1+=t)  
                    for (int64_t j1=j2; j1<j2+s && j1<n; j1+=t)  
                        for (int64_t k1=k2; k1<k2+s && k1<n; k1+=t)  
                            for (int64_t i=i1; i<i1+s && i<i2+t && i<n; i++)  
                                for (int64_t j=j1; j<j1+s && j<j2+t && j<n; j++)  
                                    for (int64_t k=k1; k<k1+s && k<k2+t && k<n; k++)  
                                        C[i*n+j] += A[i*n+k] * B[k*n+j];  
}
```

Three-level cache



Divide and Conquer

Recursive matrix multiplication

Divide-and-conquer on $n \times n$ matrices:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$
$$= \begin{bmatrix} A_{11}B_{11} & A_{11}B_{12} \\ A_{21}B_{11} & A_{21}B_{12} \end{bmatrix} + \begin{bmatrix} A_{12}B_{21} & A_{12}B_{22} \\ A_{22}B_{21} & A_{22}B_{22} \end{bmatrix}$$

8 multiply-adds of $(n/2) \times (n/2)$ matrices.

Recursive code

```
// Assume that n is an exact power of 2.
void Rec_Mult(double *C, double *A, double *B,
              int64_t n, int64_t rowsize) {
    if (n == 1)
        C[0] += A[0] * B[0];
    else {
        int64_t d11 = 0;
        int64_t d12 = n/2;
        int64_t d21 = (n/2) * rowsize;
        int64_t d22 = (n/2) * (rowsize+1);

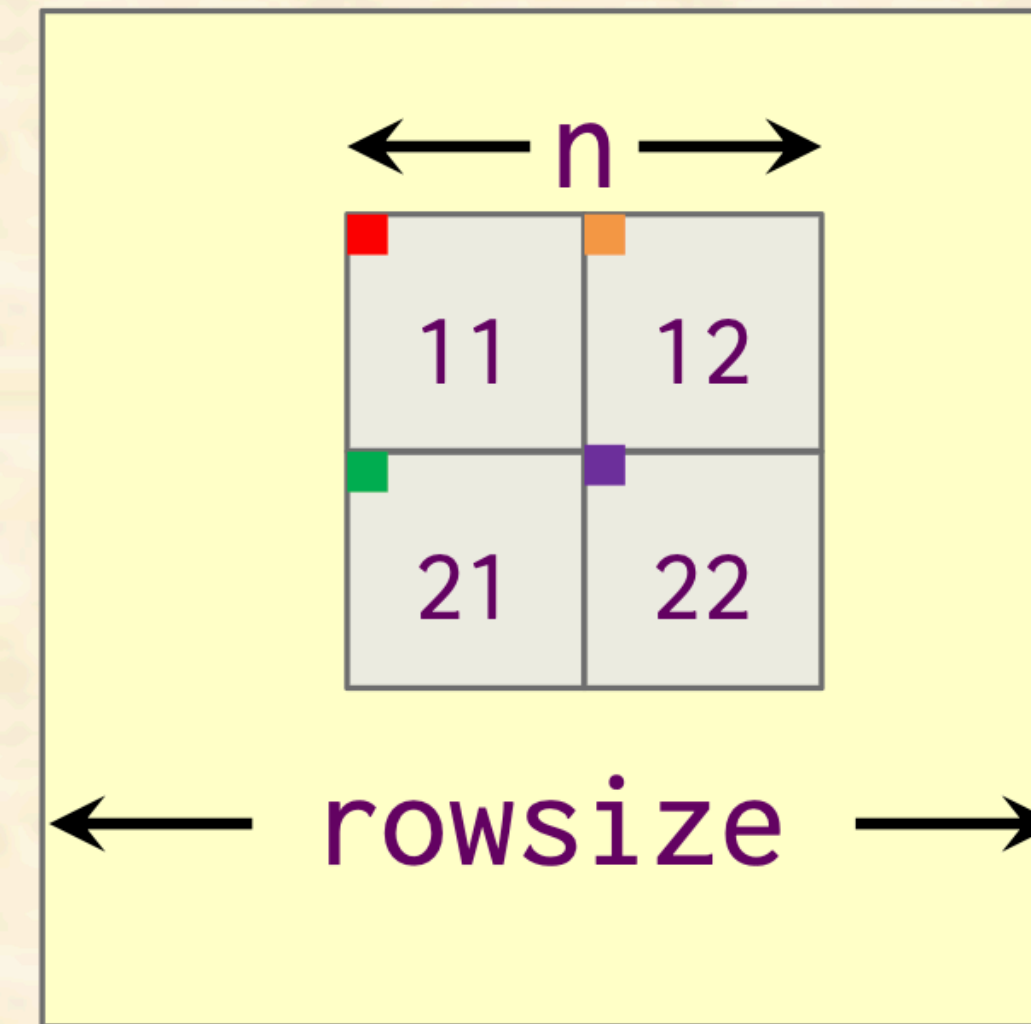
        Rec_Mult(C+d11, A+d11, B+d11, n/2, rowsize);
        Rec_Mult(C+d11, A+d12, B+d21, n/2, rowsize);
        Rec_Mult(C+d12, A+d11, B+d12, n/2, rowsize);
        Rec_Mult(C+d12, A+d12, B+d22, n/2, rowsize);
        Rec_Mult(C+d21, A+d21, B+d11, n/2, rowsize);
        Rec_Mult(C+d21, A+d22, B+d21, n/2, rowsize);
        Rec_Mult(C+d22, A+d21, B+d12, n/2, rowsize);
        Rec_Mult(C+d22, A+d22, B+d22, n/2, rowsize);
    }
}
```

Coarsen base case to overcome function-call overheads.

Recursive code

```
// Assume that n is an exact power of 2.
void Rec_Mult(double *C, double *A, double *B,
              int64_t n, int64_t rowsize) {
    if (n == 1)
        C[0] += A[0] * B[0];
    else {
        int64_t d11 = 0;
        int64_t d12 = n/2;
        int64_t d21 = (n/2) * rowsize;
        int64_t d22 = (n/2) * (rowsize+1);

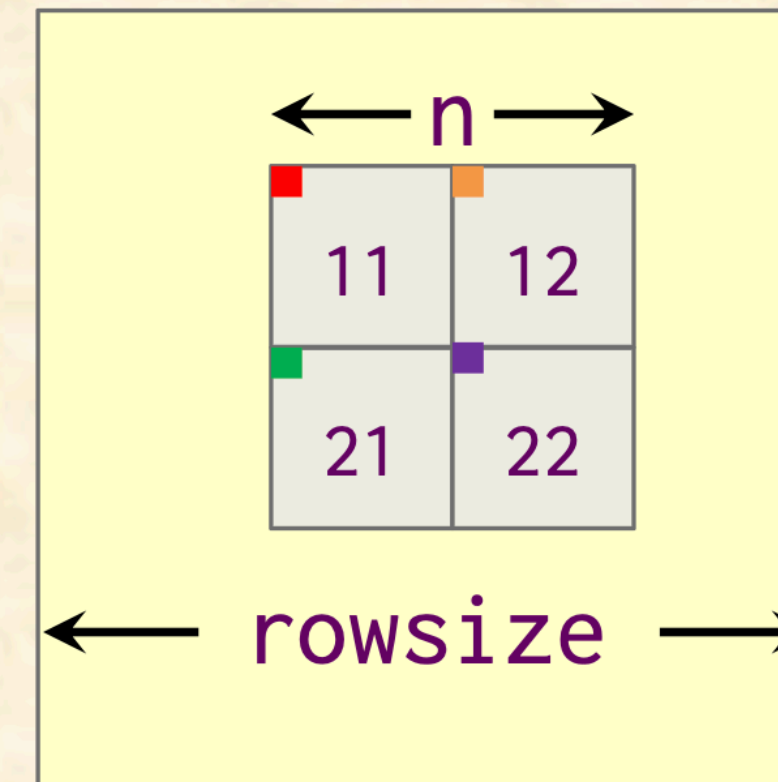
        Rec_Mult(C+d11, A+d11, B+d11, n/2, rowsize);
        Rec_Mult(C+d11, A+d12, B+d21, n/2, rowsize);
        Rec_Mult(C+d12, A+d11, B+d12, n/2, rowsize);
        Rec_Mult(C+d12, A+d12, B+d22, n/2, rowsize);
        Rec_Mult(C+d21, A+d21, B+d11, n/2, rowsize);
        Rec_Mult(C+d21, A+d22, B+d21, n/2, rowsize);
        Rec_Mult(C+d22, A+d21, B+d12, n/2, rowsize);
        Rec_Mult(C+d22, A+d22, B+d22, n/2, rowsize);
    }
}
```



Analysis of work

```
// Assume that n is an exact power of 2.
void Rec_Mult(double *C, double *A, double *B,
              int64_t n, int64_t rowsize) {
    if (n == 1)
        C[0] += A[0] * B[0];
    else {
        int64_t d11 = 0;
        int64_t d12 = n/2;
        int64_t d21 = (n/2) * rowsize;
        int64_t d22 = (n/2) * (rowsize+1);

        Rec_Mult(C+d11, A+d11, B+d11, n/2, rowsize);
        Rec_Mult(C+d11, A+d12, B+d21, n/2, rowsize);
        Rec_Mult(C+d12, A+d11, B+d12, n/2, rowsize);
        Rec_Mult(C+d12, A+d12, B+d22, n/2, rowsize);
        Rec_Mult(C+d21, A+d21, B+d11, n/2, rowsize);
        Rec_Mult(C+d21, A+d22, B+d21, n/2, rowsize);
        Rec_Mult(C+d22, A+d21, B+d12, n/2, rowsize);
        Rec_Mult(C+d22, A+d22, B+d22, n/2, rowsize);
    }
}
```



$$\begin{aligned} W(n) &= 8W(n/2) + \Theta(1) \\ &= \Theta(n^3) \end{aligned}$$

Analysis of work

$$W(n) = 8W(n/2) + \Theta(1)$$

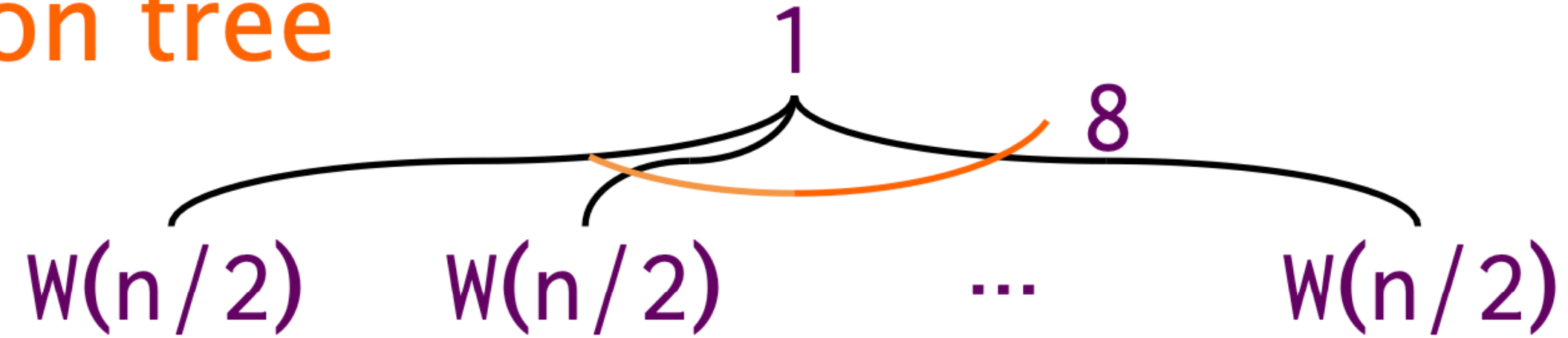
recursion tree

$W(n)$

Analysis of work

$$W(n) = 8W(n/2) + \Theta(1)$$

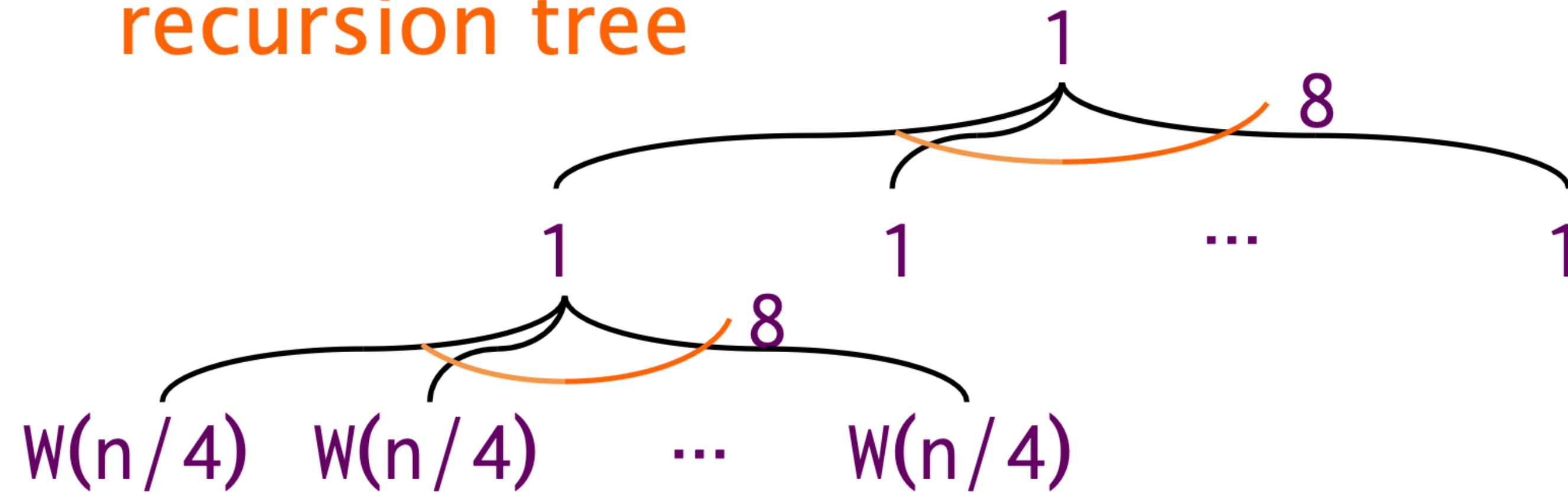
recursion tree



Analysis of work

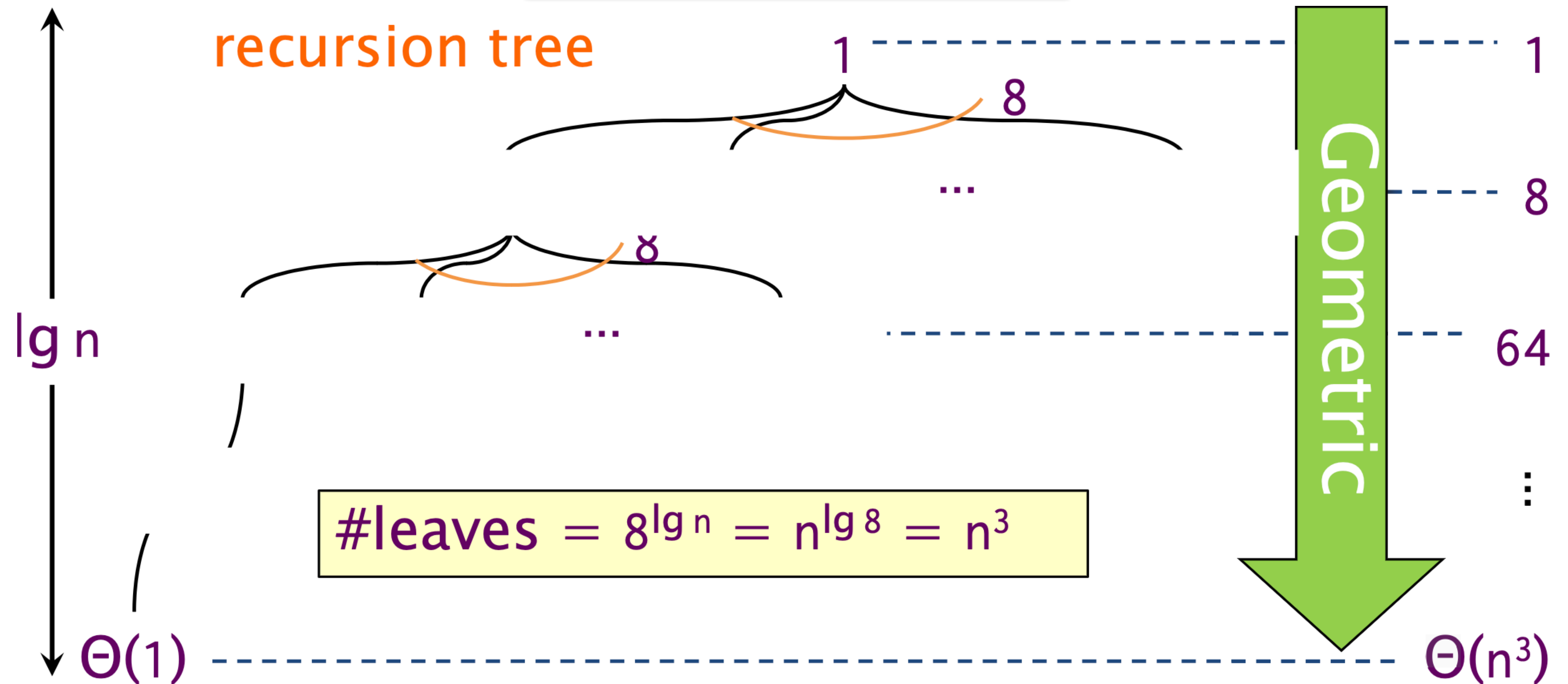
$$W(n) = 8W(n/2) + \Theta(1)$$

recursion tree



Analysis of work

$$W(n) = 8W(n/2) + \Theta(1)$$



Note: Same work as looping versions.

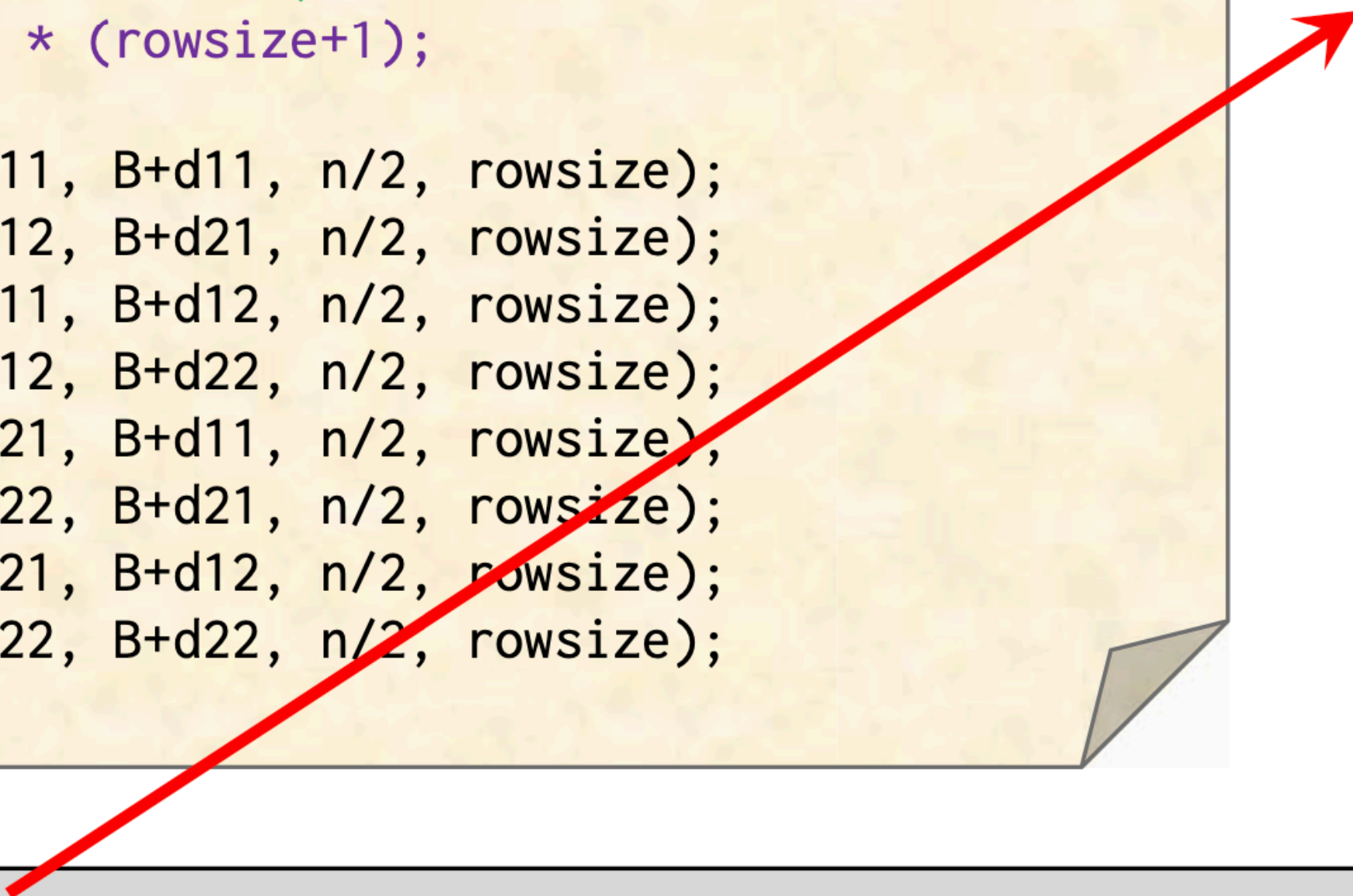
$$W(n) = \Theta(n^3)$$

Analysis of cache misses

```
// Assume that n is an exact power of 2.
void Rec_Mult(double *C, double *A, double *B,
              int64_t n, int64_t rowsize) {
    if (n == 1)
        C[0] += A[0] * B[0];
    else {
        int64_t d11 = 0;
        int64_t d12 = n/2;
        int64_t d21 = (n/2) * rowsize;
        int64_t d22 = (n/2) * (rowsize+1);

        Rec_Mult(C+d11, A+d11, B+d11, n/2, rowsize);
        Rec_Mult(C+d11, A+d12, B+d21, n/2, rowsize);
        Rec_Mult(C+d12, A+d11, B+d12, n/2, rowsize);
        Rec_Mult(C+d12, A+d12, B+d22, n/2, rowsize);
        Rec_Mult(C+d21, A+d21, B+d11, n/2, rowsize);
        Rec_Mult(C+d21, A+d22, B+d21, n/2, rowsize);
        Rec_Mult(C+d22, A+d21, B+d12, n/2, rowsize);
        Rec_Mult(C+d22, A+d22, B+d22, n/2, rowsize);
    }
}
```

Submatrix
Caching
Lemma



$$Q(n) = \begin{cases} \Theta(n^2/\mathcal{B}) & \text{if } n^2 < c\mathcal{M} \text{ for suff. small const } c \leq 1, \\ 8Q(n/2) + \Theta(1) & \text{otherwise.} \end{cases}$$

Analysis of cache misses

$$Q(n) = \begin{cases} \Theta(n^2/\mathcal{B}) & \text{if } n^2 < c\mathcal{M} \text{ for suff. small const } c \leq 1, \\ 8Q(n/2) + \Theta(1) & \text{otherwise.} \end{cases}$$

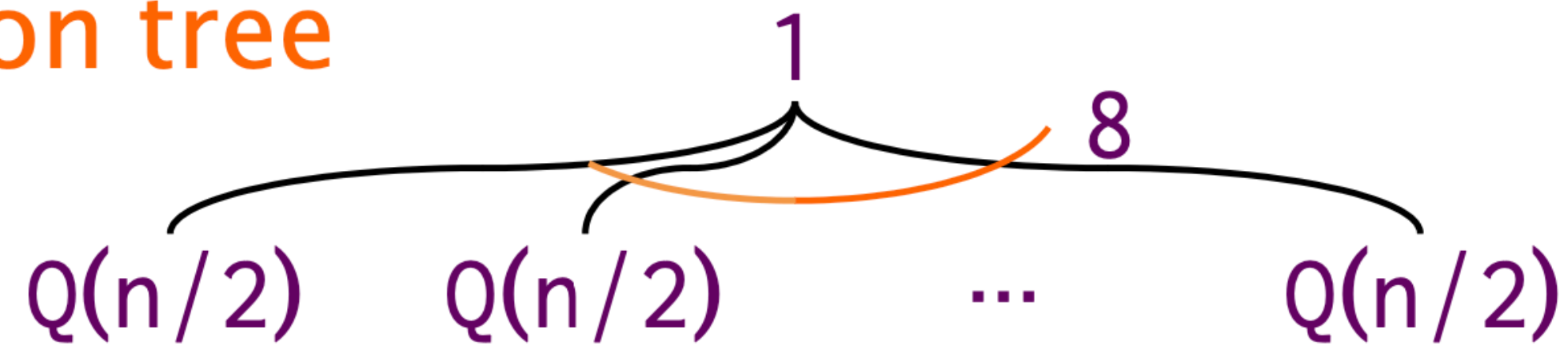
recursion tree

$Q(n)$

Analysis of cache misses

$$Q(n) = \begin{cases} \Theta(n^2/\mathcal{B}) & \text{if } n^2 < c\mathcal{M} \text{ for suff. small const } c \leq 1, \\ 8Q(n/2) + \Theta(1) & \text{otherwise.} \end{cases}$$

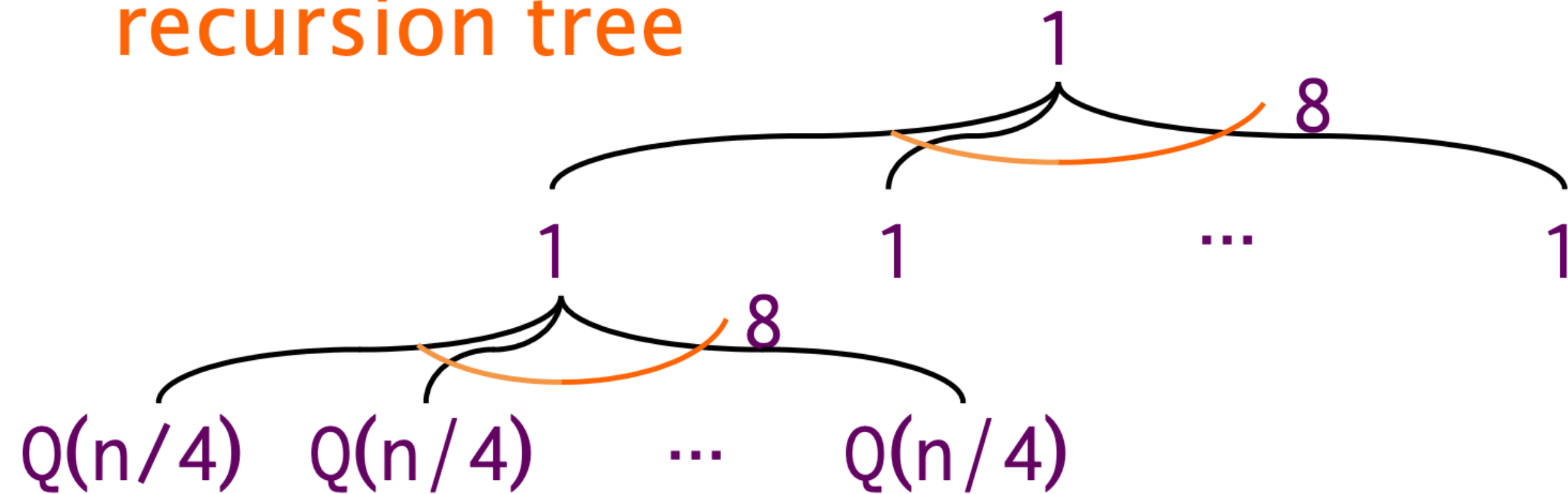
recursion tree



Analysis of cache misses

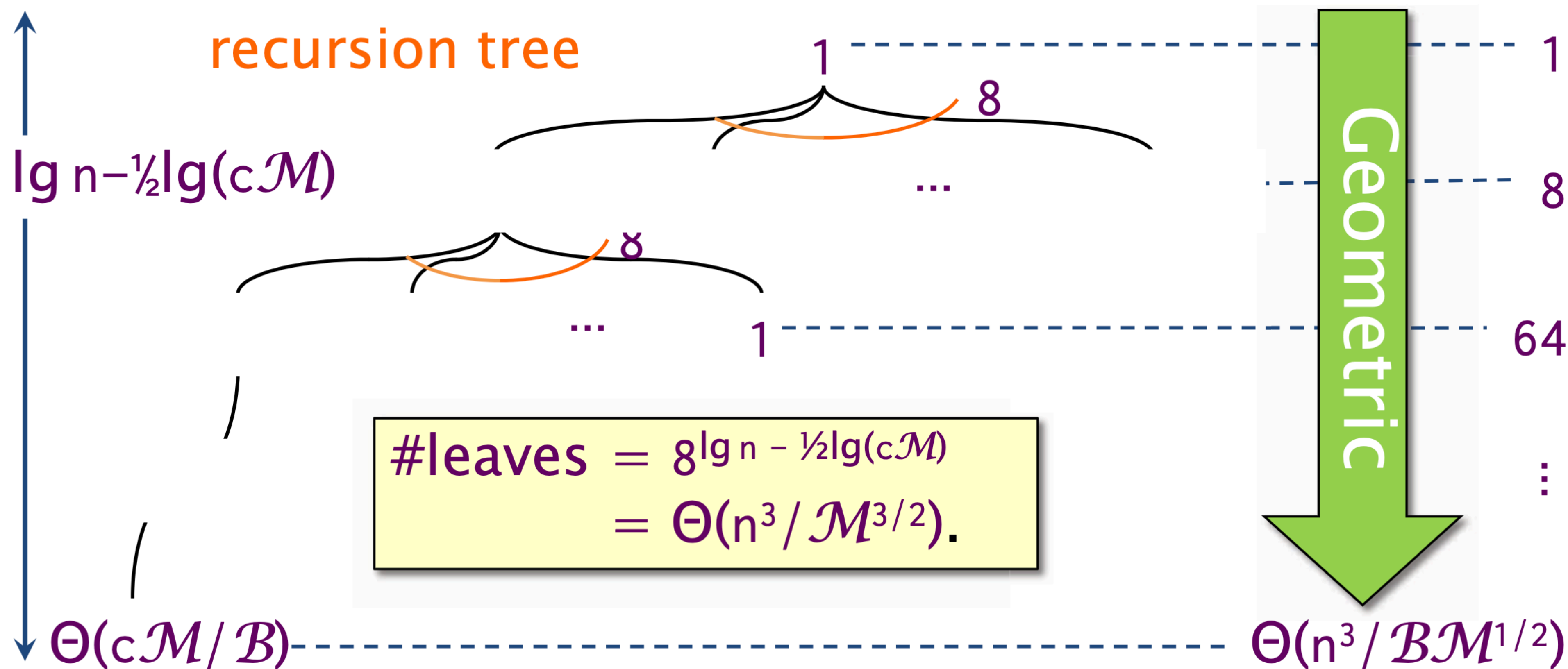
$$Q(n) = \begin{cases} \Theta(n^2/\mathcal{B}) & \text{if } n^2 < c\mathcal{M} \text{ for suff. small const } c \leq 1, \\ 8Q(n/2) + \Theta(1) & \text{otherwise.} \end{cases}$$

recursion tree



Analysis of cache misses

$$Q(n) = \begin{cases} \Theta(n^2/\mathcal{B}) & \text{if } n^2 < c\mathcal{M} \text{ for suff. small const } c \leq 1, \\ 8Q(n/2) + \Theta(1) & \text{otherwise.} \end{cases}$$



Same cache misses as with tiling!

$$Q(n) = \Theta(n^3 / \mathcal{B}\mathcal{M}^{1/2})$$

Efficient cache-oblivious algorithms

- No tuning parameters.
- No explicit knowledge of cache sizes.
- Handle multilevel caches automatically (asymptotically optimally).
- Good in multiprogrammed environments (see: cache-adaptive algorithms).

